POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY

PTIT



INFORMATION SYSTEM ANALYSIS AND DESIGN

COURSE REPORT

ATELIER FASHION

Ecommerce Information System

Requirements Identification, Requirements Analysis, and System Design

Lecturer / Giang vien: Tran Dinh Que

Student / Sinh vien: Vu Dinh Hieu

Student ID / MSV: B23DCCE036

Hanoi, 2026

Abstract

Fashion ecommerce requires the same size and colour identity to remain consistent from product discovery to warehouse picking and post-purchase service. This report specifies Atelier Fashion, a fictional direct-to-consumer retailer transitioning from disconnected catalogue, order, and inventory records to an integrated information system. The analysis establishes business activities, stakeholders, information needs, functional and nonfunctional requirements, priorities, and business rules. UML models describe actor goals, workflow responsibilities, transaction lifecycles, and domain relationships. The proposed design separates product designs from sellable variants, temporary carts from inventory reservations, and order fulfilment from payment and refund status. It uses immutable order snapshots, atomic stock allocation, authenticated provider notifications, and explicit return inspection to address overselling and inconsistent financial records. The report defines acceptance scenarios and requirement traceability for later implementation. It is a design study, not an implemented or performance-tested production system.

Contents

Abstract **i**

1 Introduction and Method **1**

1.1 Problem and purpose 1

1.2 Evidence and limitations 1

1.3 Analytical approach and theoretical foundation 2

1.4 Scope assumptions 2

2 Requirements Identification **3**

2.1 Business background 3

2.2 Current business activities and information 3

2.3 Stakeholders and needs 4

2.4 Information catalogue 5

2.5 Problems, causes, and proposed responses 6

2.6 Proposed system, boundary, and objectives 7

2.6.1 Feasibility and delivery approach 7

2.7 Elicitation and validation plan 8

2.8 Functional requirements and priorities 8

2.9 Nonfunctional requirements 10

2.10 Business rules and invariants 11

3 Requirements Analysis **13**

3.1 Actors and use case scope 13

3.2 Use case catalogue 14

3.3 Detailed use case specifications 15

3.3.1 UC01: Browse fashion catalogue 15

3.3.2 UC02: Manage account 16

3.3.3 UC03: Manage variant cart 17

3.3.4 UC04: Place order . . . 17
3.3.5 UC05: Reconcile payment event . . . 19
3.3.6 UC06: Track order . . . 19
3.3.7 UC07: Cancel order . . . 19
3.3.8 UC08: Request return or exchange . . . 20
3.3.9 UC09: Maintain catalogue . . . 20
3.3.10 UC10: Receive and adjust stock . . . 20
3.3.11 UC11: Fulfil paid order . . . 20
3.3.12 UC12: Reconcile delivery event . . . 21
3.3.13 UC13: Inspect return and allocate exchange . . . 21
3.3.14 UC14: Process refund . . . 21
3.3.15 UC15–UC18: Supporting services . . . 21
3.4 Operational use case views . . . 22
3.5 Supporting activity and swimlane models . . . 25
3.6 Order lifecycle and transaction states . . . 28
3.7 Domain concepts and analysis class models . . . 31
3.8 Cross-model consistency . . . 39

4 System Design 40
4.1 Architecture and responsibility allocation . . . 40
4.2 Checkout transaction and interaction design . . . 42
4.3 Provider callback and compensation design . . . 44
4.4 Relational mapping and data integrity . . . 46
4.5 Stock and money examples . . . 48
4.6 Interface behaviour and access design . . . 48
4.7 Deployment, recovery, and operational reporting . . . 49

5 Traceability and Validation 52
5.1 Validation approach and evidence status . . . 52
5.2 Functional requirement traceability . . . 52
5.3 Nonfunctional verification plan . . . 55
5.4 Critical scenario walkthroughs . . . 56
5.4.1 Last unit and multi-line atomicity . . . 56
5.4.2 Payment capture versus hold expiry . . . 57
5.4.3 Cancellation versus packing . . . 57

5.4.4 Partial returns, discounts, and duplicate receipts 57
5.5 Requirement quality review and change control 57

6 Conclusion and Limitations **58**

A Visual Paradigm Modelling and Reproduction Guide **59**
A.1 Native project and workflow 59
A.2 Purpose, content, and construction by diagram 59
A.3 Local automation and report rebuild 63

B Source Use and Assumption Register **64**
B.1 Use of the supplied examples 64
B.2 Theory and commercial references 64
B.3 Assumptions to confirm 64

References **65**

List of Figures

3.1 Customer services and the external payment boundary, created in Visual Paradigm. . . . 14

3.2 Product discovery with guarded alternatives and explicit merge nodes for retries. . . . 16

3.3 Checkout, stock allocation, hosted payment, and verified event processing by responsibility. . . . 18

3.4 Catalogue, warehouse, and delivery responsibilities. . . . 23

3.5 Customer service, finance, management, and access administration. . . . 24

3.6 Transaction notices, optional reviews, and manual financial reconciliation. . 25

3.7 Receiving saleable stock with validation and duplicate protection. . . . 26

3.8 Return inspection and controlled restoration of saleable stock. . . . 27

3.9 Commercial resolution after inspection: refund or reserved exchange. . . . 28

3.10 Order fulfilment lifecycle; money and returns have separate states. . . . 29

3.11 Catalogue, size charts, SKU variants, and warehouse inventory. . . . 32

3.12 Customers, carts, committed order lines, and address snapshots. . . . 34

3.13 Stock reservations are explicit commitments and are independent of carts. . 36

3.14 Independent payment, shipment, return, and exchange concepts. . . . 38

4.1 Application modules, durable work processing, and external service adapters. 41

4.2 Successful checkout: commit order, stock holds, and durable work before the provider call. . . . 43

4.3 Durable callback receipt, guarded processing, and optional late-payment compensation. . . . 45

4.4 Deployment topology, provider callbacks, and backups. . . . 50

List of Tables

2.1 Business activities and information dependencies 3
2.2 Stakeholders, responsibilities, and needs 4
2.3 Information catalogue and ownership 5
2.4 Problem analysis 6
2.5 Functional requirement baseline 8
2.6 Nonfunctional requirements and verification methods 10
2.7 Business rule baseline 11

3.1 Actor goals and requirement coverage 14
3.2 Order transition conditions and effects 29

4.1 Design responsibilities and boundaries 42
4.2 Core relational entities and constraints 46

5.1 Complete functional traceability and acceptance outcomes 52
5.2 Nonfunctional requirement evidence to collect before release 55

A.1 Diagram construction and review guide 60

Chapter 1

Introduction and Method

1.1 Problem and purpose

An online clothing purchase commits to a specific garment variant, not merely a product photograph. A shirt design can have several colour and size combinations, each with its own stock availability. An information system that records only the design cannot reliably decide whether a selected variant can be sold, picked, or exchanged. The purpose of this report is to identify and analyse the requirements for an integrated fashion ecommerce system and translate those requirements into a coherent design.

The proposed system is called *Atelier Fashion Ecommerce*. Atelier Fashion is a fictional retailer that owns its merchandise and sells directly to customers. Its commercial objective is to support dependable purchases and after-sales service while giving staff a consistent view of catalogue, inventory, orders, and financial events. The study emphasises correctness at the boundaries between business processes: checkout and payment, payment and packing, and return inspection and stock restoration.

1.2 Evidence and limitations

Three student reports supplied in `third_party` were inspected: the fashion investigation by Lê Nhữ Quỳnh, a marketplace report by Chu Tuyet Nhi, and a retail report by Phùng Thu Hồồng. Their shared sections establish a useful coverage baseline: business investigation, stakeholder needs, information analysis, problems, proposed system, requirements, use cases, activity and swimlane diagrams, traceability, and validation. The marketplace's independent sellers and payouts and the other retail report's expiry-oriented inventory are not transferred to this single-retailer fashion case. Neither example is treated as proof of UML correctness or a binding assessment rubric.

No interviews, warehouse observations, production measurements, or institution-specific marking rubric were provided. Accordingly, the as-is situation is a plausible scenario for analysis; every numerical target and policy below is a proposed assumption requiring stakeholder approval before implementation. External documentation supports modelling principles and integration patterns, not claims that a real organisation already uses the proposed design. This distinction prevents imagined survey findings or achieved benefits

from being presented as evidence.

1.3 Analytical approach and theoretical foundation

Requirements identification establishes the business problem, system boundary, stakeholder goals, and information that must be available. Requirements analysis resolves ambiguities and expresses observable behaviour through use cases, workflows, domain concepts, and rules. System design then introduces technical responsibilities, persistence structures, interfaces, and transaction mechanisms. A database table is therefore not automatically an analysis class, and an internal service is not automatically an external actor.

UML 2.5.1 is the normative reference for UML notation[1]. This report applies its concepts as follows. Actors represent roles outside the software boundary. Use cases describe goals that yield observable value. An `include` relationship points from the including use case to mandatory reused behaviour; it does not express chronological order. Actor generalisation means the specialised role can participate in the general role's interactions. Activities model control and responsibility; mutually exclusive decision branches carry guards, and alternative paths rejoin at merge nodes. State machines describe valid event-driven changes for a particular object. Class associations state domain links and multiplicities rather than arbitrary data flow.

Shopify's public model illustrates the practical distinction between a parent product and a size/colour variant, with variant-level commercial information and inventory links[2, 3]. This supports the choice to make SKU identity central to Atelier Fashion. It does not require adopting Shopify or copying its complete data schema. Stripe's documented payment and event behaviours motivate separate payment status, callback verification, and retry handling[4, 5, 6]. A production payment provider must still be selected for the retailer's operating market.

1.4 Scope assumptions

The first release supports one retailer, one warehouse, VND prices, and one shipment per order. Visitors may browse; authenticated customers may maintain a cart, check out, inspect their own orders, cancel eligible orders, and request returns or exchanges. Online prepaid payment is supported through a hosted provider interface. Staff maintain catalogue data, receive stock, pack orders, inspect returns, reconcile refunds, and review reports. Size charts are provided as reference information; automated body-measurement prediction is outside scope.

The retailer proposes a fourteen-day return request window from confirmed delivery, subject to item condition and disclosed product exclusions. An exchange is limited to an alternative size or colour of the same product with the same agreed item price. Exchanges to a different product or price are handled as a return/refund followed by a new purchase. These are case policies, not asserted universal ecommerce rules or legal conclusions. Guest checkout, cash on delivery, seller onboarding, seller settlement, international tax calculation, manufacturing, and procurement accounting are outside the first release.

Chapter 2

Requirements Identification

2.1 Business background

Atelier Fashion sells clothing and accessories through an online storefront. Merchandising staff organise seasonal collections, categories, material descriptions, care instructions, photographs, and size charts. Each product design has one product identifier. Each sellable size/colour combination has one unique SKU. A blue shirt in size M and the same blue shirt in size L are distinct inventory items even when their listed prices match.

The retailer earns revenue from merchandise sales. Unlike a marketplace, it owns stock and does not calculate commissions or seller payouts. Garment suppliers deliver stock to the warehouse; shipping and payment providers perform external services. Customers need accurate fit and availability information, while internal staff require unambiguous item identities and evidence of what was promised at purchase.

2.2 Current business activities and information

The assumed current operation uses spreadsheets, social messages, and provider portals. This scenario establishes design problems without claiming a measured field study. Table 2.1 distinguishes current work from the capabilities proposed to improve it.

Table 2.1: Business activities and information dependencies

Activity	Assumed current operation	Proposed support
Catalogue publication	Staff share product descriptions, images, and size lists in separate files.	Maintain product designs and explicit SKU matrices; publish only complete active variants.
Customer information	Addresses are copied from messages into order sheets.	Store customer-owned addresses and immutable delivery snapshots per order.
Product discovery	Customers browse listings and ask staff about size availability.	Filter products by category, size, colour, and price; display availability for the selected SKU.

Activity	Assumed current operation	Proposed support
Cart preparation	Customers assemble selections, sometimes using product names alone.	Record exact SKU and quantity; clarify that carts do not reserve stock.
Checkout	Staff check stock and calculate totals manually.	Recalculate authoritative prices, allocate stock atomically, and preserve the agreed total.
Payment verification	Staff compare order sheets with a provider portal.	Receive verified payment events and reconcile unresolved transactions.
Order fulfilment	Staff pick from text descriptions and record dispatch separately.	Produce SKU pick lists; block dispatch until verified payment and packing checks.
Delivery tracking	Customers ask staff to look up tracking references.	Store one shipment reference and validated carrier updates per order.
Returns and exchanges	Requests, parcel receipt, refund approval, and stock updates are disconnected.	Link requests to order lines; inspect goods before restoring stock or dispatching replacements.
Management reporting	Teams manually consolidate sales and stock sheets.	Report SKU sales, stock, returns, and confirmed financial events with defined measures.

2.3 Stakeholders and needs

Stakeholder status does not imply direct use of the software. Suppliers affect stock quality and receiving data but do not have accounts in this scope. Payment and shipping providers are external software actors because their interfaces exchange information with the system.

Table 2.2: Stakeholders, responsibilities, and needs

Stakeholder	Responsibility	Need and expected value
Visitor / customer	Choose garments and purchase	N01: Accurate size/colour information, transparent totals, secure account access, and understandable order/return status.
Merchandiser	Maintain sellable catalogue	N02: One source for product attributes, SKU uniqueness, images, size charts, prices, and publication state.
Warehouse staff	Receive, pick, pack, and inspect	N03: Available and committed stock by SKU, reliable pick lists, and controlled return stock restoration.

Stakeholder	Responsibility	Need and expected value
Customer service	Resolve cancellations and returns	N04: Purchase snapshots, eligibility rules, delivery evidence, and auditable decisions.
Finance staff	Reconcile payments and refunds	N05: Provider references, verified monetary events, outstanding exceptions, and prevention of duplicate refunds.
Manager	Evaluate business performance	N06: Consistent reporting definitions, SKU-level sales, return reasons, and seasonal stock visibility.
Administrator	Control access and configuration	N07: Role permissions, secure staff accounts, configuration versioning, and audit evidence.
Payment provider	Process funds externally	N08: Order reference, agreed amount/currency, idempotent requests, and authenticated event exchange.
Shipping provider	Transport parcels externally	N09: Recipient snapshot, shipment reference, parcel details, and tracking event exchange.
Garment supplier	Supply physical goods	N10: Consistent SKU references and discrepancy records; coordination remains outside the software.

2.4 Information catalogue

Master information changes over time; transaction information records a commitment or event; derived information is calculated from identified source records. This distinction explains why a profile change must not rewrite a historical delivery address and why an edited product price must not change an existing order.

Table 2.3: Information catalogue and ownership

Information	Principal data and classification	Owner and control
Product and variant	Product title, category, material, size-chart version, colour images; SKU, size, colour, price, active state. Master.	Merchandiser; unique SKU and unique product/size/colour combination; retain referenced discontinued variants.
Customer and address	Account identity, contact information, password hash, address book. Master/private.	Customer; restricted staff access; address snapshots on orders are separate.
Inventory	SKU, on-hand quantity, reserved quantity, stock movement reason and reference. Operational.	Warehouse; atomic updates and audit; no negative available balance.

Information	Principal data and classification	Owner and control
Cart	Customer, exact SKU, requested quantity, revision. Temporary intention.	Customer; no stock hold; prices and availability rechecked at checkout.
Order and lines	Owner, timestamp, recipient/address snapshot, SKU/title/size/colour snapshots, quantities, prices, discounts, delivery charge, total. Transaction.	Order service; immutable commercial snapshot after confirmation.
Reservation	Order/SKU quantities, expiry, active/consumed/released state. Transaction.	Inventory service; unique allocation and once-only release/consumption.
Payment and refund	Provider object reference, amount, currency, status, event identifiers, refund reason. Transaction.	Finance; trusted provider confirmation and idempotency.
Shipment	Tracking reference, carrier, dispatch time, delivery events. Transaction.	Warehouse/carrier integration; ownership checks and event validation.
Return and exchange	Original order line, requested quantity/reason, receipt and inspection result, replacement SKU or refund allocation. Transaction.	Customer service and warehouse; cumulative quantity checks.
Audit and reports	Actor, action, time, before/after reference; daily aggregates. Audit/derived.	Administrator/manager; restricted audit access; reproducible report definitions.

2.5 Problems, causes, and proposed responses

Table 2.4: Problem analysis

Problem	Cause	Consequence and response	Needs
P01 Wrong garment variant	Product names replace size/colour SKU identity.	Mis-picks and exchanges; use the same SKU in browsing, cart, order, and pick list.	N01–03
P02 Overselling	Availability check and stock allocation occur separately.	Two buyers obtain the last unit; allocate all order SKUs atomically at checkout.	N01,03
P03 Historical data drift	Orders reference editable prices and profile addresses.	Disputes over totals/address; preserve purchase and delivery snapshots.	N01,04

Problem	Cause	Consequence and response	Needs
P04 False paid status	Browser redirects or manual flags are treated as proof.	Unpaid dispatch; verify provider events and reconcile pending payment objects.	N03,05
P05 Premature restocking	Refund approval is treated as physical goods receipt.	Damaged or absent items become sellable; restock only after inspection.	N03–05
P06 Duplicate financial work	Retries and partial requests lack unique references and quantity caps.	Repeated charge/refund or over-return; use idempotency and cumulative allocations.	N04,05
P07 Inconsistent reports	Sales, cash, and refunds are combined without definitions.	Managers cannot reconcile performance; define and label each metric.	N05,06

2.6 Proposed system, boundary, and objectives

The proposed system integrates a public storefront, authenticated customer services, staff operations, and controlled provider adapters. Inside the boundary are catalogue publication, variant carts, checkout, stock reservations, order coordination, return decisions, refund requests, access control, and reporting. Outside are physical manufacture, parcel transport, the provider's settlement process, and supplier purchasing contracts. Provider status is recorded and verified rather than invented by the application.

The proposed objectives are to preserve exact SKU identity through each transaction, prevent committed stock from exceeding sellable stock, block fulfilment without verified payment, maintain traceable monetary events, and make post-purchase decisions auditable. Success is assessed against acceptance scenarios rather than unsupported claims of a percentage improvement over a nonexistent measured baseline.

2.6.1 Feasibility and delivery approach

A modular web application with a transactional relational database is technically appropriate for a single retailer because stock and order changes can share a local transaction. Hosted payment pages reduce the application's direct handling of payment credentials. Operational feasibility depends on staff agreeing on SKU naming, barcode practice, return inspection, and responsibility for exception queues. Economic feasibility cannot be proved without supplier quotations and transaction volumes; a later budget must include hosting, provider fees, support, security review, and staff training.

Delivery proceeds through catalogue/SKU cleanup, customer ordering and payment, warehouse workflows, then returns and reporting. Each phase requires approved requirements, migration reconciliation, and acceptance review. A pilot warehouse count establishes opening stock; spreadsheets are archived and reconciled before becoming read-only. No estimated implementation cost or achieved return on investment is asserted.

2.7 Elicitation and validation plan

The proposed elicitation methods are interviews with merchandising, warehouse, finance, and customer-service representatives; observation of receiving, picking, and returns; inspection of anonymised order sheets and payment references; and walkthroughs of difficult scenarios such as the last unit being purchased twice. Customer testing should focus on selecting unavailable sizes, understanding size charts, and distinguishing pending payment from confirmed purchase. These are planned activities, not completed research.

Open decisions include the chosen payment/carrier interfaces, excluded return categories, shipping fee rules, and retention schedules. Before development, the manager and relevant staff approve a requirement baseline. Changes receive a version, reason, impact assessment, and updated links to use cases, rules, data, and acceptance scenarios.

2.8 Functional requirements and priorities

Each requirement below expresses a system obligation. *Must* identifies a release condition; *Should* may be deferred without breaking the core transaction; *Could* is optional. Out-of-scope functions are recorded separately rather than disguised as low-priority obligations. The fourth MoSCoW category, *Won't in this release*, includes guest checkout, cash on delivery, marketplace seller onboarding/settlement, manufacturing, multi-warehouse allocation, and perishable expiry tracking. These exclusions define this release's boundary; expansion requires a new requirement baseline and corresponding model changes.

Table 2.5: Functional requirement baseline

ID	Requirement	Priority
FR01	The system shall search/filter active products by keyword, category, size, colour, and price, and display selected-SKU availability and size-chart information.	Must
FR02	The system shall register/authenticate customers and allow them to manage their own contact data and addresses; staff accounts shall be provisioned by administrators.	Must
FR03	The system shall store exact SKU quantities in customer carts, support add/update/remove, and state that a cart does not hold stock.	Must
FR04	The system shall revalidate the cart, address, prices, and delivery charge; show the final total for consent; and create at most one order for one checkout request key.	Must
FR05	The system shall reserve all required SKU quantities in the same transaction as order creation; reject the whole checkout if any SKU lacks stock.	Must
FR06	The system shall initiate hosted online payment with the agreed order total/currency and persist the provider reference using an idempotent request.	Must

ID	Requirement	Priority
FR07	The system shall verify provider events, deduplicate deliveries, validate amount/currency/order linkage, and reconcile uncertain or late outcomes.	Must
FR08	The system shall show a customer only their own order snapshots, payment state, shipment tracking, and return/refund progress.	Must
FR09	The system shall permit customer-service/customer cancellation before packing begins; release allocations once and request any required refund independently.	Must
FR10	The system shall accept eligible per-line returns/exchanges, enforce the request window and remaining quantities, and record decisions with reasons.	Must
FR11	The system shall allow merchandisers to maintain products, SKU variants, images, size charts, prices, and publication state without rewriting historical orders.	Must
FR12	The system shall record stock receipts/adjustments per SKU with quantity, reason, staff identity, and movement reference.	Must
FR13	The system shall generate SKU pick lists, verify packing, create one shipment per order, and consume reservations when goods are dispatched.	Must
FR14	The system shall validate and store carrier events, protect verified delivery from older events, and support authorised permanent non-delivery closure with independent refund and physical receipt controls.	Must
FR15	The system shall record return receipt and inspection; restore only saleable quantities and allocate equal-price replacement stock before confirming an exchange.	Must
FR16	The system shall request refunds against captured funds, cap cumulative amounts, prevent duplicate submissions, and confirm completion only from provider evidence.	Must
FR17	The system shall provide daily order, captured-payment, confirmed-refund, SKU sales, and stock reports with explicit measures and date filters.	Should
FR18	The system shall restrict actions by role, enforce customer/resource ownership, and audit sensitive changes and financial/stock events.	Must
FR19	The system shall notify customers of confirmed order, shipment, return decision, and refund outcomes without rolling back business transactions when notification fails.	Should
FR20	The system may allow customers with delivered purchases to submit one moderated review per purchased product per order.	Could

2.9 Nonfunctional requirements

The following numerical targets are proposed acceptance conditions. They are not benchmarks already achieved. Provider/network time is reported separately from internal processing time.

Table 2.6: Nonfunctional requirements and verification methods

ID	Target	Verification method
NFR01	At 100 concurrent active sessions and a 10,000-variant catalogue, internal search response p95 shall be at most 2 s and checkout allocation p95 at most 3 s.	Representative load test; publish test hardware, workload, and p95 results.
NFR02	The production service shall target 99.5% monthly availability, excluding announced maintenance; external provider incidents shall be reported separately.	Monitor successful core journeys and calculate downtime over the calendar month.
NFR03	Concurrent purchases, cancellations, expiry jobs, and returns shall preserve stock and money invariants without lost updates.	Concurrency and retry scenarios; database reconciliation.
NFR04	Sensitive actions shall enforce server-side role/ownership checks, staff MFA, secure password hashing, TLS, and verified provider signatures. ASVS guides the security review.	Access-control, injection, session, and callback security tests; no certification claim[7].
NFR05	No raw card credentials shall be stored. Contact/address data shall be minimised, access-controlled, and excluded from routine logs.	Storage/log inspection and permission review.
NFR06	Core browsing, cart, checkout, and return forms shall target WCAG 2.2 AA, with keyboard navigation, labelled controls, visible focus, and non-colour status cues.	Manual keyboard/screen-reader checks and automated accessibility checks[8].
NFR07	The service shall target an RPO of 15 min and an RTO of 4 h using backups and database log recovery.	Restore drill; confirm event/outbox replay and payment reconciliation.
NFR08	The application shall centralise configurable policies, separate provider adapters, and record correlation identifiers across transaction steps.	Review module interfaces; trace one checkout and one return through logs.

ID	Target	Verification method
NFR09	Staff shall be able to trace every stock adjustment, cancellation, return decision, and refund to an authorised actor or validated external event.	Reconstruct sampled event histories; detect missing references.

2.10 Business rules and invariants

Table 2.7: Business rule baseline

ID	Rule
BR01	SKU is unique across the retailer. A product/size/colour tuple identifies at most one variant. Only published active variants are purchasable.
BR02	Cart membership does not reserve inventory. For each SKU, available stock equals saleable on-hand stock minus active reserved stock; all three values are nonnegative.
BR03	Checkout reserves every order line atomically or reserves none. The initial payment hold lasts 15 minutes. Expiry closes the order and releases active holds once; a late captured payment creates a compensation/refund exception rather than reviving the order.
BR04	Unit prices, line discounts, SKU descriptions, recipient address, and delivery charge are snapshotted after customer consent. A changed quote requires renewed consent. Amounts use integer VND, not floating-point arithmetic.
BR05	Order total equals the sum of quantity times agreed unit price less allocated line discount, plus delivery charge. A line discount cannot exceed its gross line value. Listed prices are treated as tax-inclusive in this academic case; tax compliance requires separate jurisdictional review.
BR06	Payment is successful only after trusted provider verification of its reference, order, amount, and currency. A browser success page alone cannot authorise packing. Duplicate events have no duplicate business effect.
BR07	Only a paid, confirmed order can enter packing. Customer cancellation is permitted only in PendingPayment or Confirmed. A version/lock check resolves cancellation versus packing so only one eligible transition succeeds.
BR08	At dispatch, consume the reservation and reduce on-hand stock by the same quantity. Cancellation/expiry releases reserved stock without reducing on-hand stock. Once dispatched, use a return process rather than cancellation.

ID	Rule
BR09	Return requests must reference delivered order lines and be submitted within the assumed 14-day policy window. Completed and active return allocations together cannot exceed the quantity delivered for a line. Rejected/withdrawn allocations are released.
BR10	Refund approval and returned-stock restoration are distinct events. Only received, inspected, saleable goods increase saleable on-hand stock. Damaged goods are recorded separately.
BR11	An exchange replaces a line with another size/colour of the same product at the same agreed price. Staff allocate replacement stock after inspection before confirming the exchange; unavailable replacements require customer choice of refund or withdrawal.
BR12	Confirmed plus pending refund commitments cannot exceed captured funds, and line refund commitments cannot exceed the refundable discounted value of eligible quantities. Delivery charge is refunded for full pre-dispatch cancellation or permanent non-delivery closure under BR15; post-delivery item returns exclude it. Partial item refunds use deterministic per-unit discount allocation.
BR13	Payment, fulfilment, return, and refund status are separate. Delivered remains the fulfilment fact even when some items are returned or refunded. RefundRequested is not evidence that money has reached the customer.
BR14	A staff role confers only its assigned permissions. Customers can read or change only their own resources; suppliers have no direct software access.
BR15	Permanent non-delivery closure requires verified carrier evidence and Customer Service authorisation while the order is Shipped. It schedules a full captured-amount refund, including delivery. Restocking requires a unique physical carrier-return receipt and saleability inspection; lost goods are not restocked. Temporary delivery attempts do not trigger closure.

Chapter 3

Requirements Analysis

3.1 Actors and use case scope

A Visitor browses public product information. A Customer specialises that role and performs authenticated shopping and post-purchase actions. Staff actors are Merchandiser, Warehouse Staff, Customer Service, Finance Staff, Manager, and Administrator. A person may hold more than one staff role, but permissions are checked for each action. Payment Provider and Shipping Provider exchange messages across the software boundary. The database, scheduler, and application services are internal design elements rather than external actors.

Figure 3.1 shows the customer/payment view. Place order includes checkout validation, stock reservation, and payment initiation in its successful flow. Payment event reconciliation is a distinct externally triggered interaction: the customer's browser does not confirm the transfer of funds. The generalisation triangle points to Visitor so Customer inherits public browsing. An operations view complements these customer goals with catalogue, inventory, fulfilment, returns, refunds, reports, and administration.

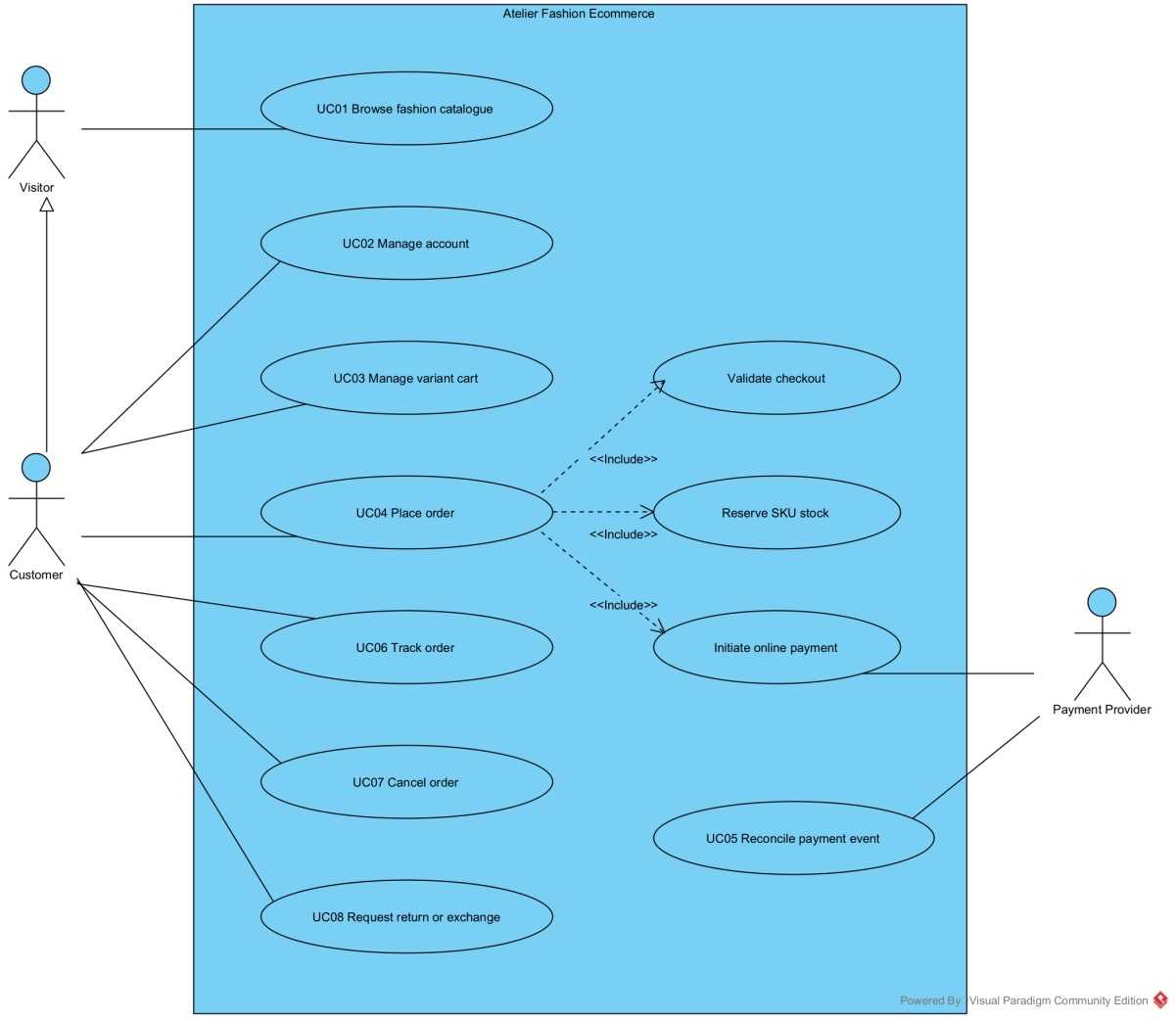


Figure 3.1: Customer services and the external payment boundary, created in Visual Paradigm.

3.2 Use case catalogue

Table 3.1: Actor goals and requirement coverage

ID	Goal	Primary actor	Requirements
UC01	Browse fashion catalogue	Visitor, Customer	FR01
UC02	Manage account	Customer	FR02
UC03	Manage variant cart	Customer	FR03
UC04	Place order	Customer	FR04–06
UC05	Reconcile payment event	Payment Provider; Finance Staff for exceptions	FR07
UC06	Track order	Customer	FR08

ID	Goal	Primary actor	Requirements
UC07	Cancel order	Customer, Customer Service	FR09
UC08	Request return or exchange	Customer	FR10
UC09	Maintain catalogue	Merchandiser	FR11
UC10	Receive and adjust stock	Warehouse Staff	FR12
UC11	Fulfil paid order	Warehouse Staff	FR13
UC12	Reconcile delivery event	Shipping Provider, Customer Service	FR14
UC13	Inspect return and allocate exchange	Warehouse Staff, Customer Service	FR15
UC14	Process refund	Finance Staff, Payment Provider	FR16
UC15	View management reports	Manager, Finance Staff	FR17
UC16	Administer access	Administrator	FR18
UC17	Receive transaction notifications	Customer	FR19
UC18	Submit purchased-product review	Customer	FR20

3.3 Detailed use case specifications

The specifications distinguish preconditions from validation performed during a use case. A precondition is a state assumed when the interaction begins; malformed input, insufficient stock, and provider failures remain explicit exception flows. The minimum guarantee states what remains true even when the goal is not achieved.

3.3.1 UC01: Browse fashion catalogue

Goal and trigger. A Visitor or Customer finds a suitable garment and selects an exact variant. The trigger is a search, filter change, or product selection. No authentication is required to browse.

Main flow. (1) The actor enters keywords and optional category, size, colour, and price filters. (2) The system returns published products with matching variants. (3) The actor opens a product. (4) The system displays photographs, material/care information, size chart, price, and variant availability. (5) The actor chooses size and colour. (6) The system resolves the exact SKU and enables adding an available selection to the cart.

Alternatives. No matching results produce a clear empty result and editable filters. An unavailable size/colour remains visible with an explanation and another selection option. Unpublished products cannot be purchased through old links. Before authenticated cart mutation, an unauthenticated visitor is asked to sign in; public viewing does not create a

customer cart.

Postconditions. The actor can identify the selected SKU. Browsing neither allocates stock nor creates an order. A cached availability display remains advisory; UC04 performs the authoritative allocation. FR01 and BR01–02 apply.

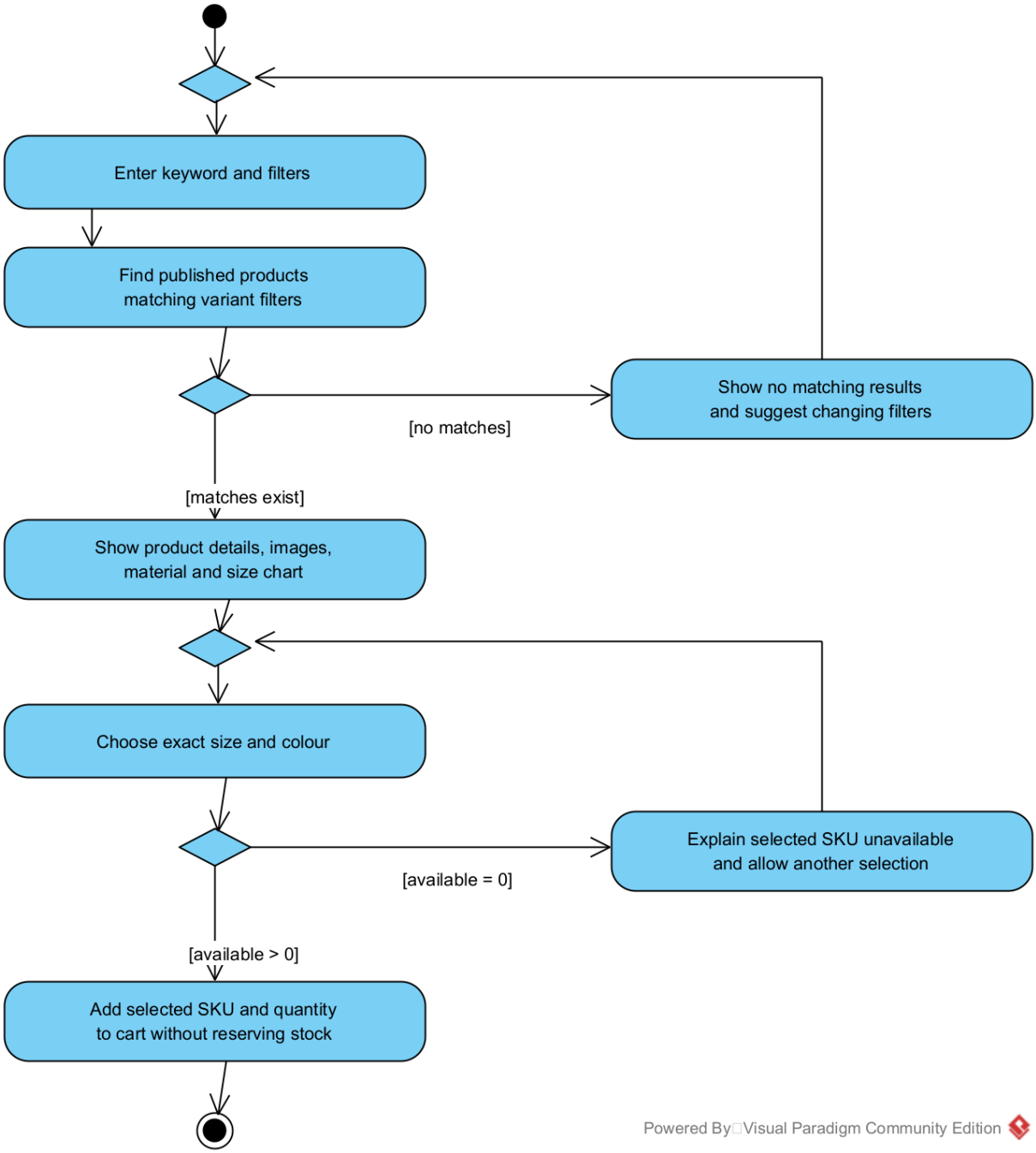


Figure 3.2: Product discovery with guarded alternatives and explicit merge nodes for retries.

3.3.2 UC02: Manage account

Actor, trigger, and conditions. A Customer requests registration, sign-in, profile maintenance, address changes, or sign-out. Profile and address changes require an au-

thenticated session owned by that customer.

Main flow. Validate registration data and email uniqueness; store a password hash; authenticate using server-side checks; display the owner's profile; validate and persist an authorised change; confirm its result. Sign-out invalidates the active session. Registration creates only customer privileges.

Exceptions and guarantees. Invalid credentials return a generic error and are rate-limited. A reused email cannot create a duplicate identity. Expired sessions require reauthentication before mutation. Address changes affect future checkout choices and never rewrite existing order snapshots. FR02, FR18, BR04, and BR14 apply.

3.3.3 UC03: Manage variant cart

Precondition and trigger. The Customer is authenticated and chooses to add, change, or remove an exact SKU.

Main flow. Resolve an active SKU; validate a positive integer quantity; add or update the unique customer-cart/SKU line; increment the cart revision; display current estimates and explain that stock is not reserved. Removal deletes the selected cart line.

Exceptions and guarantees. A stale cart revision requires refreshing rather than silently overwriting another browser's edit. Discontinued or unavailable selections receive a per-line explanation. Checkout revalidates all prices and stock even when cart display was recently refreshed. Cart changes never decrement inventory. FR03 and BR02 apply.

3.3.4 UC04: Place order

Actors and trigger. The Customer consents to the final checkout quote; Payment Provider supports online payment. The Customer is authenticated and has a nonempty cart.

Main success flow.

1. Load the customer-owned cart and requested address; calculate current item prices, discount allocation if configured, and delivery charge.
2. Present exact SKU quantities, recipient details, and the final VND total. Obtain explicit consent to that quote.
3. On submission, check the request key and quote/cart revision; lock required SKU inventory records in deterministic order.
4. Validate all variants and quantities. Create the PendingPayment order, immutable order lines/address, and active reservations in one database transaction. Record an outbound payment command.
5. Commit the transaction. Initiate payment outside the inventory transaction with a stable provider idempotency key; save the resulting provider reference.
6. Return the existing or new order reference and hosted payment instruction. The customer completes any provider authentication.
7. UC05 subsequently verifies the payment outcome. A successful eligible event moves the order to Confirmed and converts the temporary hold into a fulfilment allocation.

Alternatives and exceptions. A changed price, shipping quote, or cart revision requires

renewed consent. Insufficient stock for any line rolls back the entire order/allocation transaction. A repeated request with the same key returns the same order; changed payload with a reused key is rejected. A timeout after order creation displays PendingPayment and allows safe resumption; it does not create another order or assume that the provider failed. The system reconciles payment status before further financial action. When the hold expires, release it and close the order under BR03; a late captured payment enters a compensation queue.

Postconditions. Success creates one attributable order with exact snapshots and stock allocation. Minimum guarantee: no partial allocation, no unconsented total, no duplicate order for one request, and no dispatch authorised by a browser redirect. FR04–07 and BR02–08 apply.

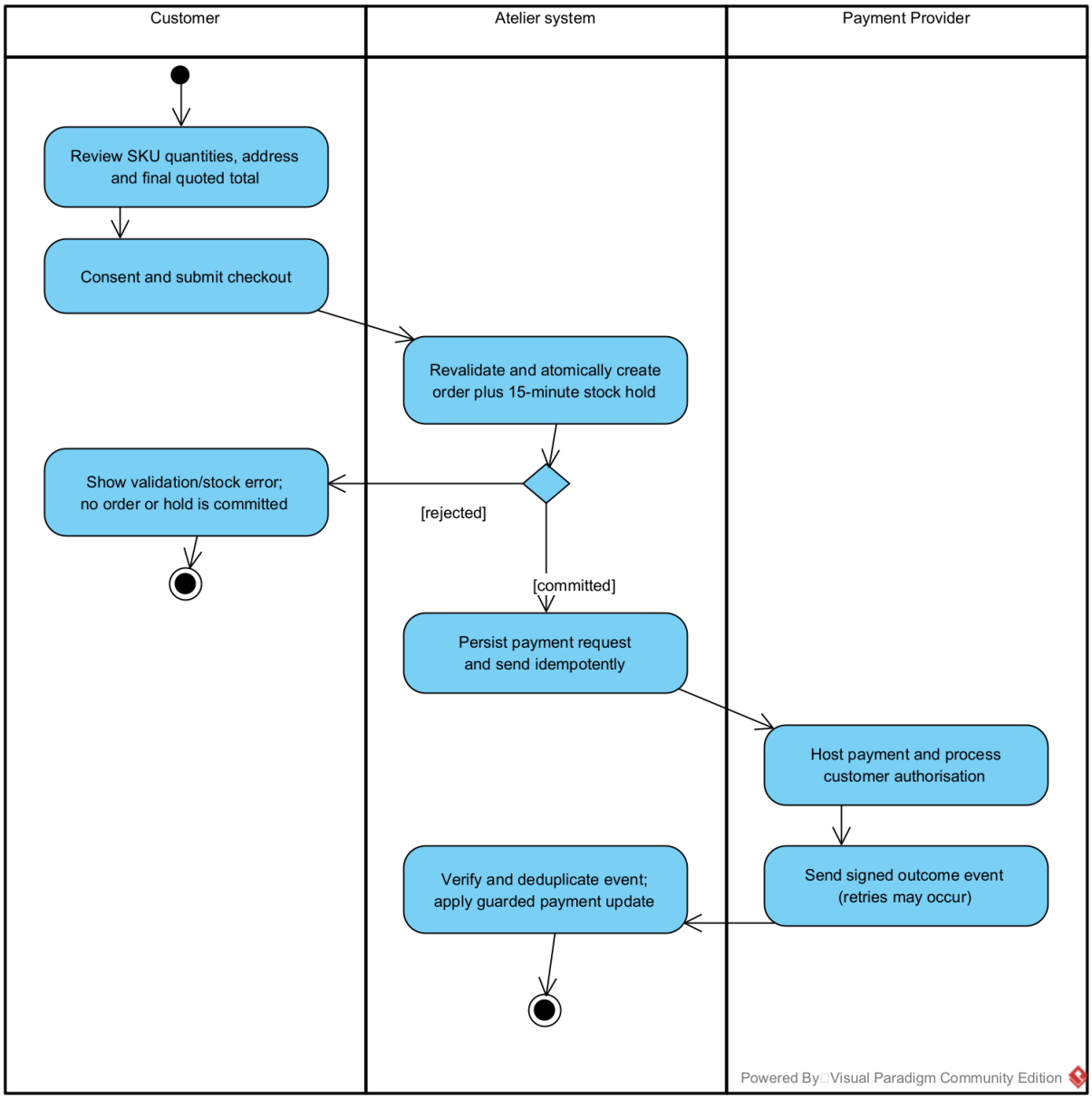


Figure 3.3: Checkout, stock allocation, hosted payment, and verified event processing by responsibility.

3.3.5 UC05: Reconcile payment event

Trigger and actor. Payment Provider sends an outcome notification. Finance Staff initiate controlled reconciliation of unresolved provider objects. An incoming unverified message is input to validate, not a trusted precondition.

Main flow. (1) Verify the signature and record the event durably. (2) Deduplicate its provider/event identifier. (3) Resolve the payment object and its order; validate reference, total, and currency against the order snapshot. (4) Lock the order/payment state. (5) If capture succeeded and the order is still eligible, record captured funds, set payment Succeeded, set order Confirmed, and retain fulfilment allocations. (6) Record processing evidence and schedule notification through an outbox.

Exceptions. Invalid signatures are rejected without changing business state. A duplicate event acknowledges previously completed work. Out-of-order events cannot regress verified success; uncertain status triggers retrieval of the authoritative provider object. Mismatched amount/currency/reference enters an exception queue and blocks fulfilment. Failure allows retry while a valid hold remains. Capture after cancellation or expiry is recorded as real money received and schedules an idempotent compensation refund; it cannot revive the closed order. Processing must not depend on event arrival order, a behaviour documented for provider integrations[5].

Guarantees. Each valid financial effect is recorded once, and only an eligible fully allocated order becomes fulfilment-ready. FR07, BR03, and BR06 apply.

3.3.6 UC06: Track order

The authenticated Customer opens an owned order. The system displays the immutable purchase/address details, payment outcome, fulfilment state, shipment reference, and separately identified return/refund progress. An unknown or other customer's order is inaccessible even if its identifier is guessed. Missing carrier updates show the last verified event and its time rather than a fabricated arrival estimate. Viewing changes no transaction state. FR08 and BR13–14 apply.

3.3.7 UC07: Cancel order

The Customer or Customer Service requests cancellation of an owned/authorised order in PendingPayment or Confirmed. The system locks the order, verifies packing has not begun, changes fulfilment to Cancelled, releases active reservations once, and records actor/reason. It then requests cancellation of an unresolved provider payment or schedules refund of captured funds. If packing acquired the transition first, cancellation is refused and the customer sees the current state. Repeated cancellation returns the existing outcome. Refund remains independently pending until verified provider completion. FR09 and BR07–08, BR12–13 apply.

3.3.8 UC08: Request return or exchange

Preconditions and trigger. The authenticated Customer chooses delivered lines from an owned order. The trigger is submission of quantities, reason, requested remedy, and optional replacement size/colour.

Main flow. Validate the policy window and disclosed exclusions; lock original line return allocations; calculate the remaining eligible quantity after prior completed and active requests; record the request and line quantities; return instructions and a reference. Customer Service reviews eligibility and records an approved/rejected decision with a reason.

Alternatives. Expired windows or excessive quantities are rejected with explanations. Concurrent overlapping requests cannot both allocate the same unit. A rejected or withdrawn request releases its provisional quantity allocation. An exchange request is a preference, not a replacement-stock guarantee; UC13 decides stock availability after receipt/inspection. No request approval restores stock or proves a refund. FR10 and BR09–11 apply.

3.3.9 UC09: Maintain catalogue

The authorised Merchandiser creates or edits a product design, links category, material, and size chart information and colour images, defines unique variant SKUs, and validates price and publication data. The system refuses duplicate SKU or product, size, and colour identity, incomplete publication, and negative prices. Referenced variants are deactivated rather than physically deleted. Updating a live price affects new quotes, not historical order lines. FR11 and BR01, BR04 apply.

3.3.10 UC10: Receive and adjust stock

Warehouse Staff identify the received SKU, count saleable units, provide a supplier receipt or adjustment reference and reason, and submit the stock movement. The system checks role, validates quantity, prevents duplicate receipt references, and atomically updates on-hand and movement history. A downward correction cannot make on-hand less than committed reservations; the discrepancy is escalated instead of silently creating negative availability. Damaged receipts are recorded outside saleable stock. The supplier remains a stakeholder, not an authenticated software actor. FR12 and BR02, BR14 apply.

3.3.11 UC11: Fulfil paid order

Warehouse Staff select a Confirmed order with verified captured payment and active allocations. The system atomically enters Packing, generates a SKU/quantity pick list, and records packing checks. Staff verify exact size/colour and recipient snapshot, then request shipment creation through the carrier adapter. On successful dispatch registration, record the shipment and consume reservations while decrementing on-hand stock. A duplicate dispatch action returns the existing shipment. Wrong-SKU scans block packing completion. Carrier timeout leaves a recoverable Packing exception; it does not consume stock twice or create a second shipment blindly. Customers cannot cancel once packing

begins in the baseline policy. FR13 and BR06–08 apply.

3.3.12 UC12: Reconcile delivery event

Shipping Provider submits tracking events. The system authenticates the message, identifies the known shipment, deduplicates the event, records its timestamp/evidence, and performs an allowed shipment/order transition. Older in-transit events cannot undo Delivered. Unknown shipments or contradictory events enter an exception queue. Temporary failed attempts allow redelivery. Customer Service may close permanent non-delivery only with verified evidence while the order is Shipped; the system records DeliveryFailed and schedules an independent full captured-amount refund including delivery. Stock remains physically outside the warehouse until a carrier return is actually received, counted, and inspected through UC10 with a unique receipt reference. Only saleable units are restored; a lost parcel produces no stock increase. This path does not use the delivered-order customer-return eligibility rule. FR14 and BR08, BR10, BR13, BR15 apply.

3.3.13 UC13: Inspect return and allocate exchange

Warehouse Staff receive the referenced parcel and record per-line inspected quantities and condition. Customer Service records whether the accepted remedy is refund or exchange. The system confirms that receipt quantities do not exceed the approved request, restores only saleable accepted units, and records damaged units separately. For an exchange, it atomically reserves the equal-price replacement SKU and creates a linked replacement fulfilment. Unavailable replacement stock prompts the Customer to choose refund or withdrawal; it does not promise nonexistent stock. Replacement dispatch follows the same SKU packing controls but creates no new customer charge. Repeated inspection references cannot restock twice. FR15 and BR09–11 apply.

3.3.14 UC14: Process refund

Finance Staff confirm an eligible refund arising from cancellation, accepted return, permanent non-delivery closure, or late-payment compensation. The system locks refund commitments, determines the remaining refundable captured amount, allocates any line-level entitlement, and persists an idempotent refund request. The provider call occurs after local commit. The system records Pending until verified provider completion then marks Succeeded and updates confirmed refunded funds. Timeouts reconcile the existing refund object before retry; failed refunds retain a visible exception and may be retried against the same entitlement. Already pending commitments cannot be spent again by a concurrent refund request. A provider refund failure does not reverse a physical warehouse inspection. FR16 and BR10, BR12–13 apply.

3.3.15 UC15–UC18: Supporting services

UC15, reports. Manager or Finance Staff select a date range and permitted measure. The system aggregates the defined order/stock/payment/refund sources and labels time-

zone and event-date basis. No-data results show an explicit empty state. Reports never equate order value with collected cash.

UC16, access. Administrator provisions staff roles, changes permissions, and revokes accounts. The system checks administrative authorisation, audits the change, invalidates revoked access, and refuses self-service customer elevation. Other business use cases still enforce their own permissions; access control is a pervasive requirement rather than an include arrow attached to every ellipse.

UC17, notifications. After a committed business event, a worker sends the relevant customer notice and records delivery attempts. Retry failures enter a notification queue. The customer may inspect the same authoritative status in UC06 even if email delivery fails.

UC18, optional reviews. An authenticated Customer with a delivered product submits a rating and comment. The system checks purchase eligibility and the unique customer/order/product review, validates content, and queues moderation before publication. Rejected content receives a moderation reason. This Could feature may be deferred.

3.4 Operational use case views

The customer view is complemented by Figures 3.4 and 3.5. Catalogue work belongs to the Merchandiser; receiving, packing, and physical inspection belong to Warehouse Staff. Customer Service participates in return decisions, while the Shipping Provider supports shipment registration and supplies delivery events. Finance Staff reconcile refunds and financial reports. The Manager reads business reports; the Administrator manages access rather than being granted every operational responsibility automatically. UC17 notifications and UC18 optional reviews are supporting services specified in the catalogue; they do not change the stock and money commitments shown in the core views.

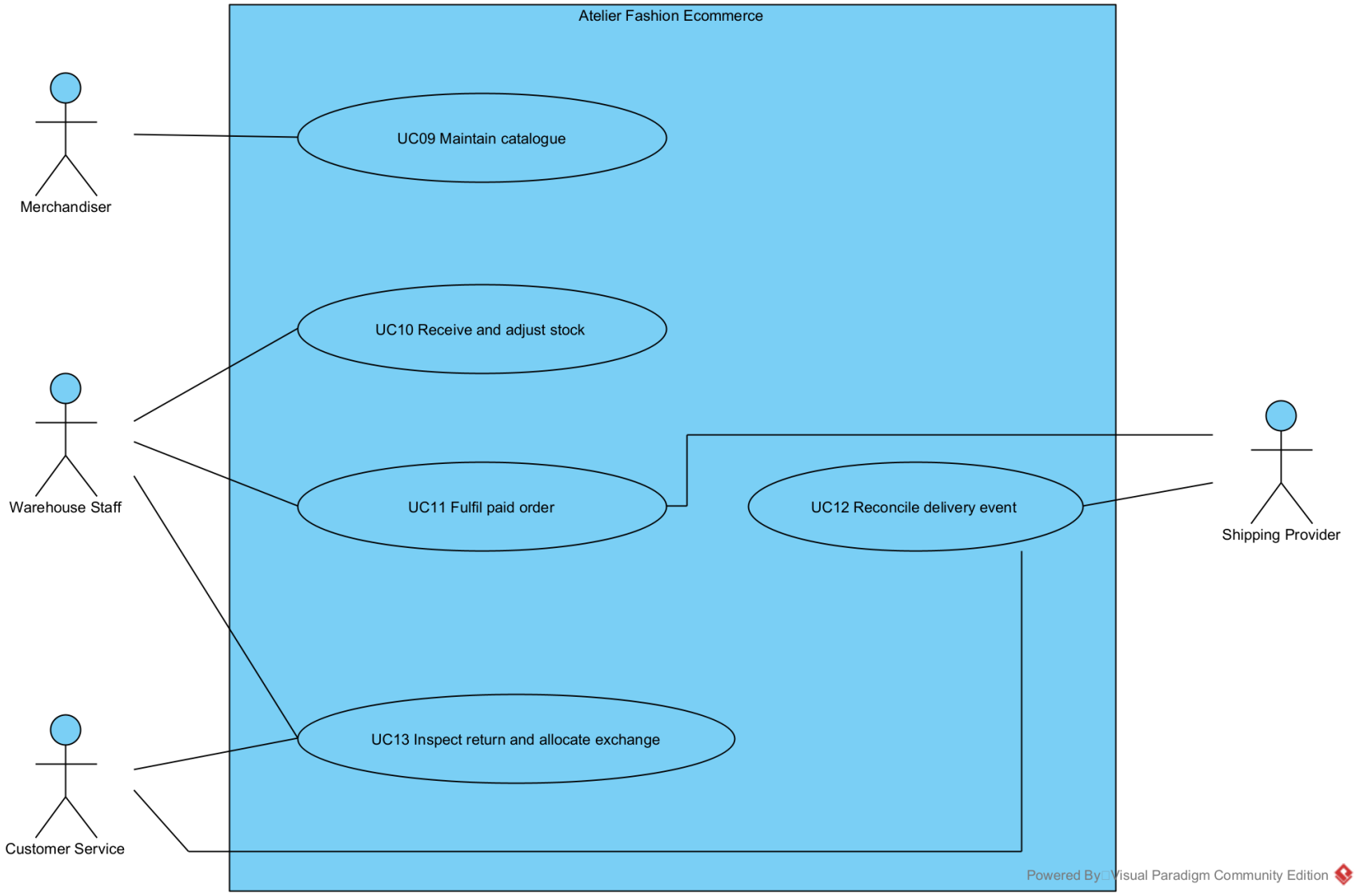


Figure 3.4: Catalogue, warehouse, and delivery responsibilities.

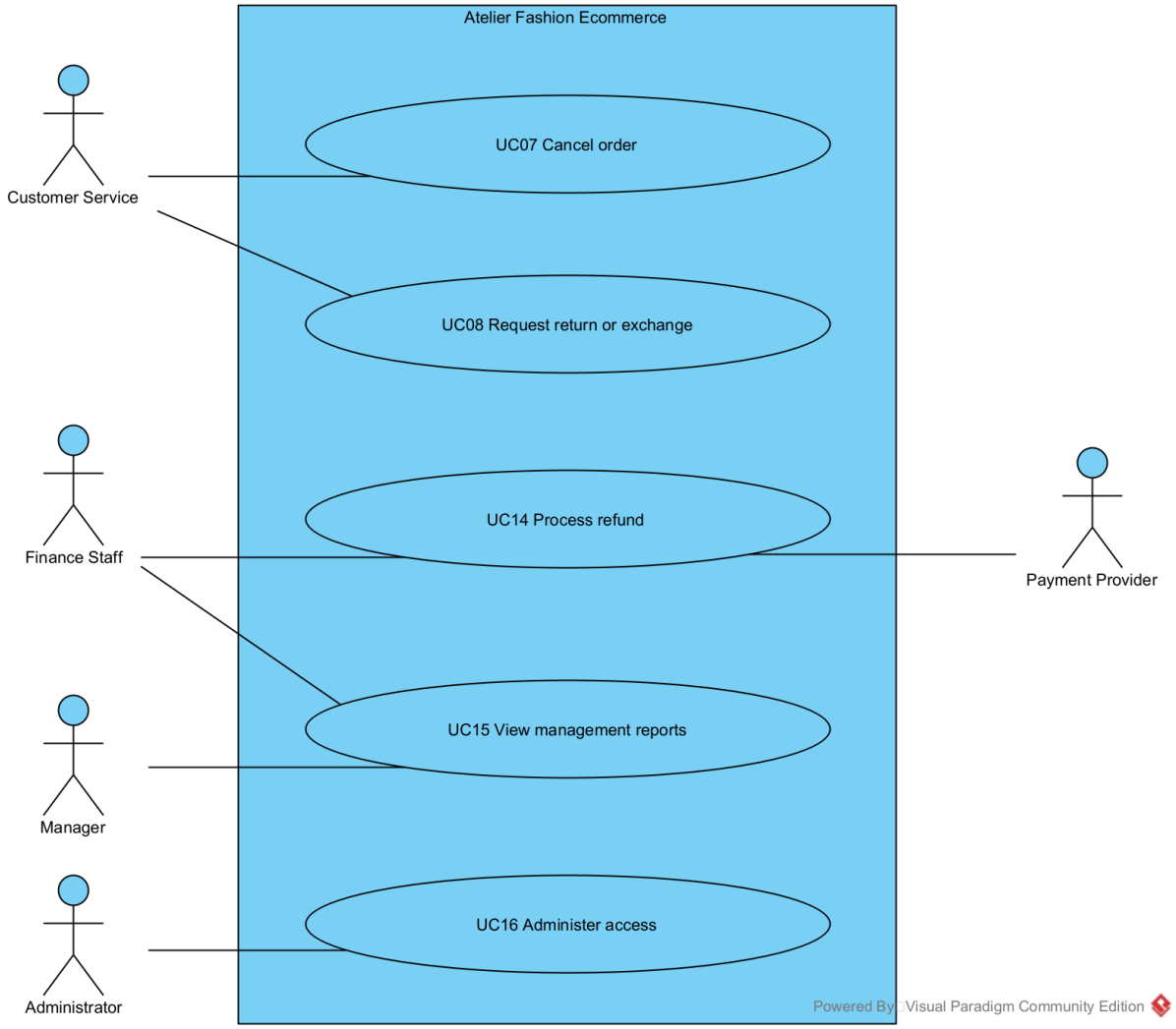


Figure 3.5: Customer service, finance, management, and access administration.

Figure 3.6 makes the supporting goals explicit in the native model. A Customer receives committed transaction notifications and may submit an eligible purchased-product review. Finance Staff can reconcile unresolved payment events through UC05; the provider remains the normal external event source. Review publication is optional under the Could priority.

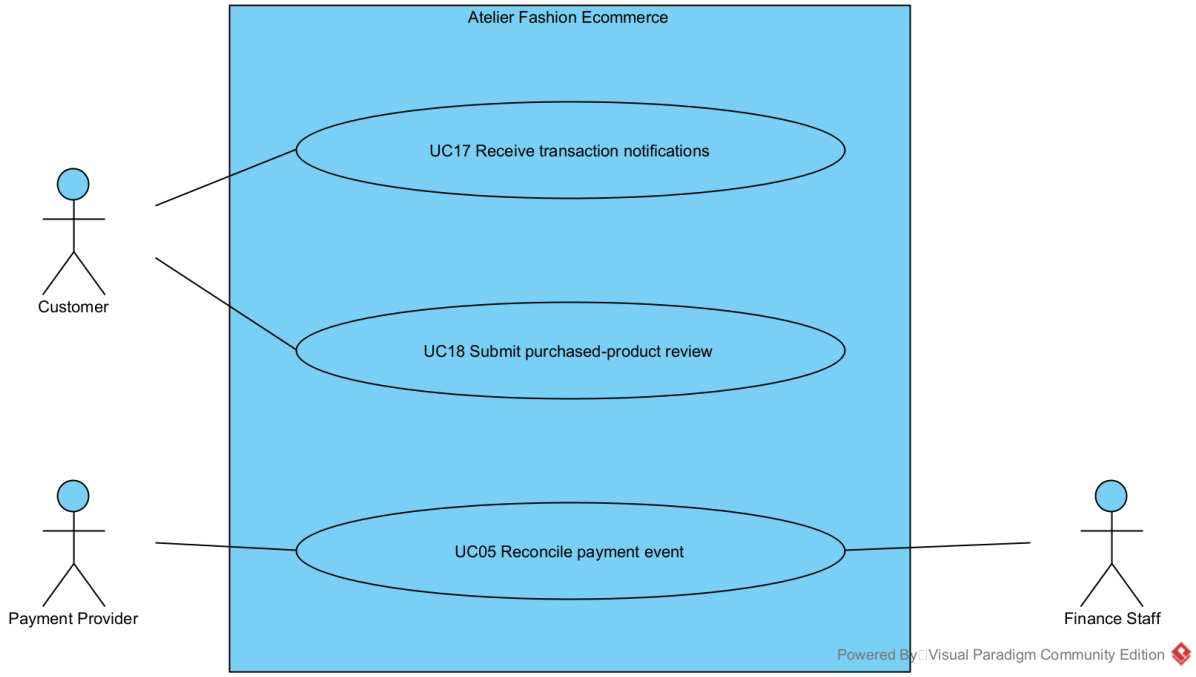


Figure 3.6: Transaction notices, optional reviews, and manual financial reconciliation.

3.5 Supporting activity and swimlane models

Figure 3.7 develops the stock receipt obligation. A receipt key prevents a repeated submission from becoming another stock increase. The actual count separates damaged from saleable units. Only validated new data reaches the atomic posting action; the alternative reports invalid input or the already recorded result.

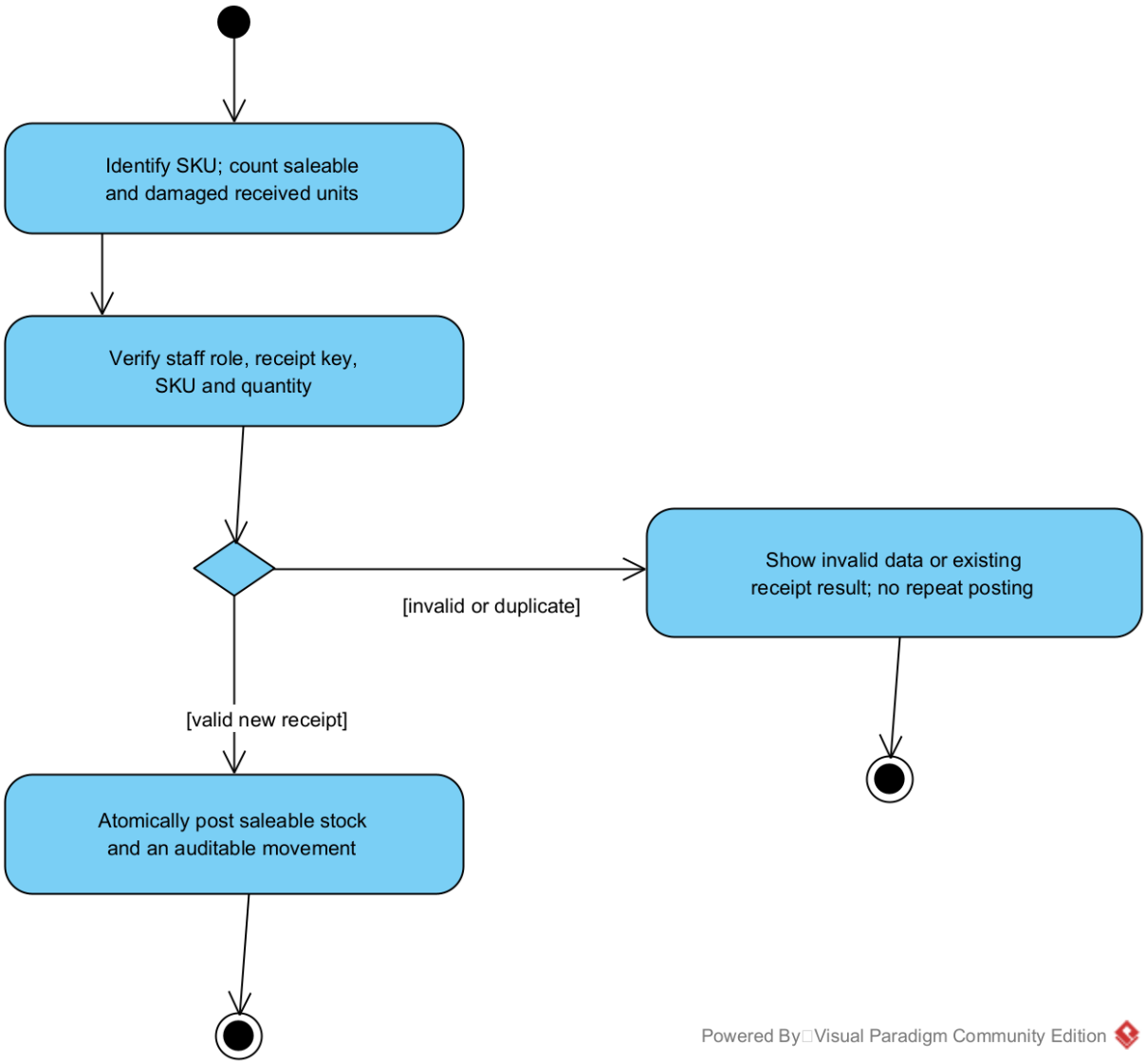


Figure 3.7: Receiving saleable stock with validation and duplicate protection.

Figures 3.8 and 3.9 divide post-purchase service into physical verification and commercial resolution. The inspection workflow begins with an already approved request. Warehouse Staff verify the parcel and condition, then the system checks quantities and receipt identity before committing stock changes. A refund decision alone cannot produce this physical receipt evidence. A disputed receipt ends the current processing attempt without committing stock; staff must resolve the discrepancy before another attempt.

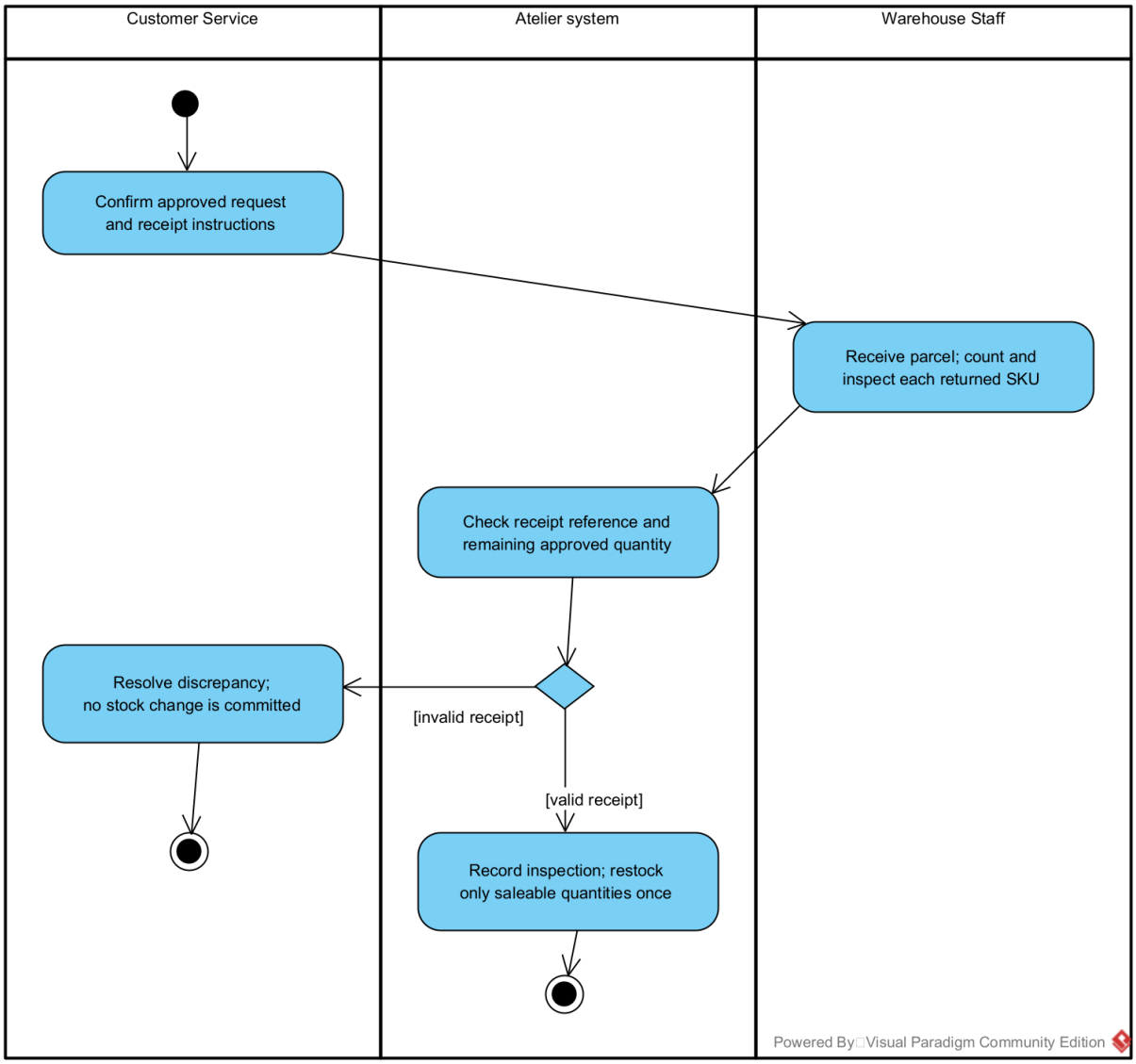


Figure 3.8: Return inspection and controlled restoration of saleable stock.

Once goods have been accepted and inspected, an exchange preference is resolved against actual replacement stock. Figure 3.9 reserves the requested equal-price replacement before confirming linked fulfilment. Unavailable stock produces a pending customer decision; the final node terminates that processing attempt, not the entire ReturnRequest lifecycle. A later refund choice resumes UC14, while withdrawal closes the exchange preference. The refund branch schedules financial work and does not assert that the provider has already completed it.

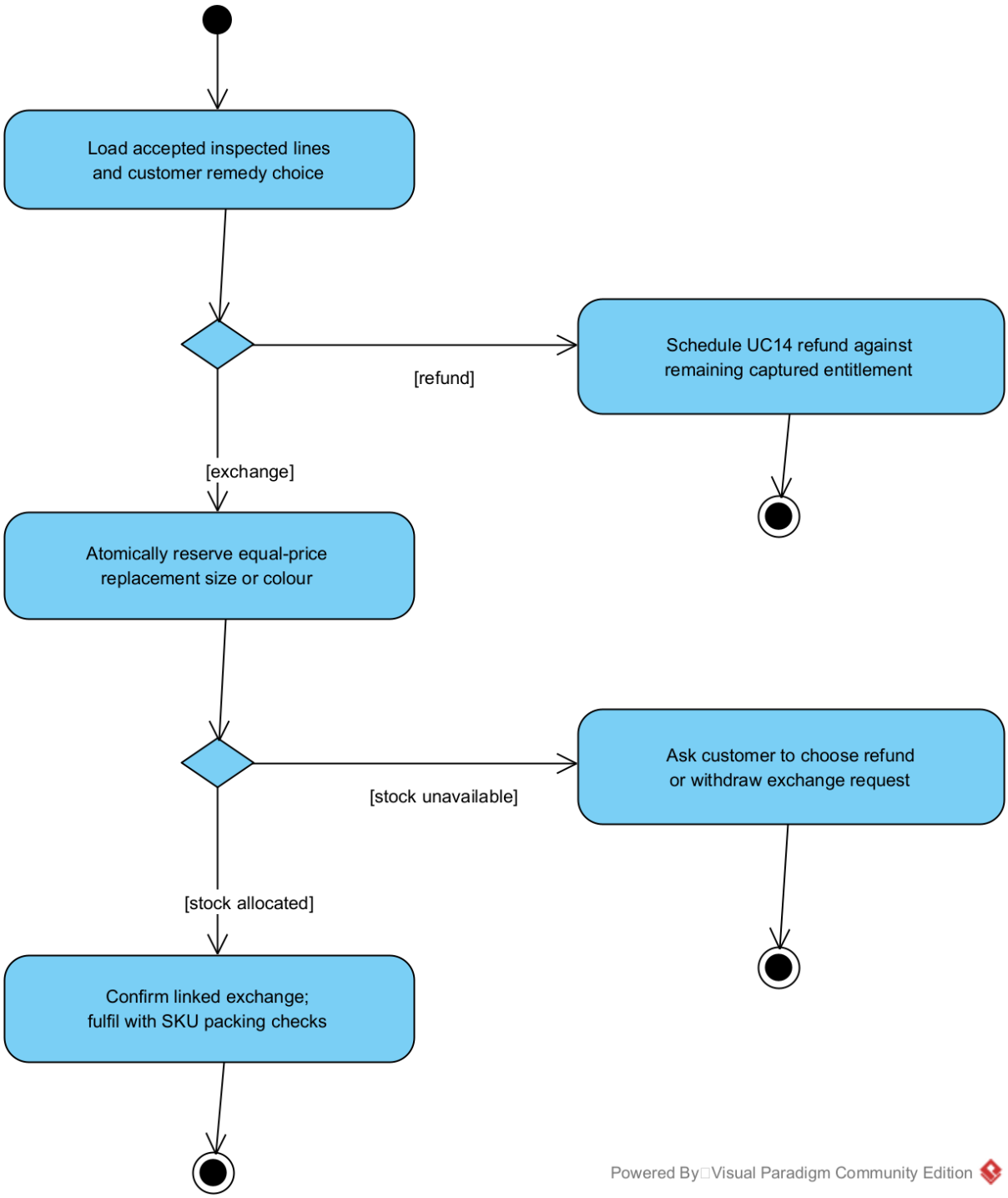


Figure 3.9: Commercial resolution after inspection: refund or reserved exchange.

3.6 Order lifecycle and transaction states

Figure 3.10 describes *order fulfilment*, not every financial or post-purchase fact. The initial transition enters PendingPayment only after checkout has committed the order and its reservations. Confirmed means trusted capture and valid allocation are both established. Packing blocks customer cancellation in the first-release policy. Dispatch consumes stock allocation; verified delivery records the delivery fact. Cancelled, Expired, Delivered, and

DeliveryFailed are persistent business states. They do not need to destroy the order object or erase evidence.

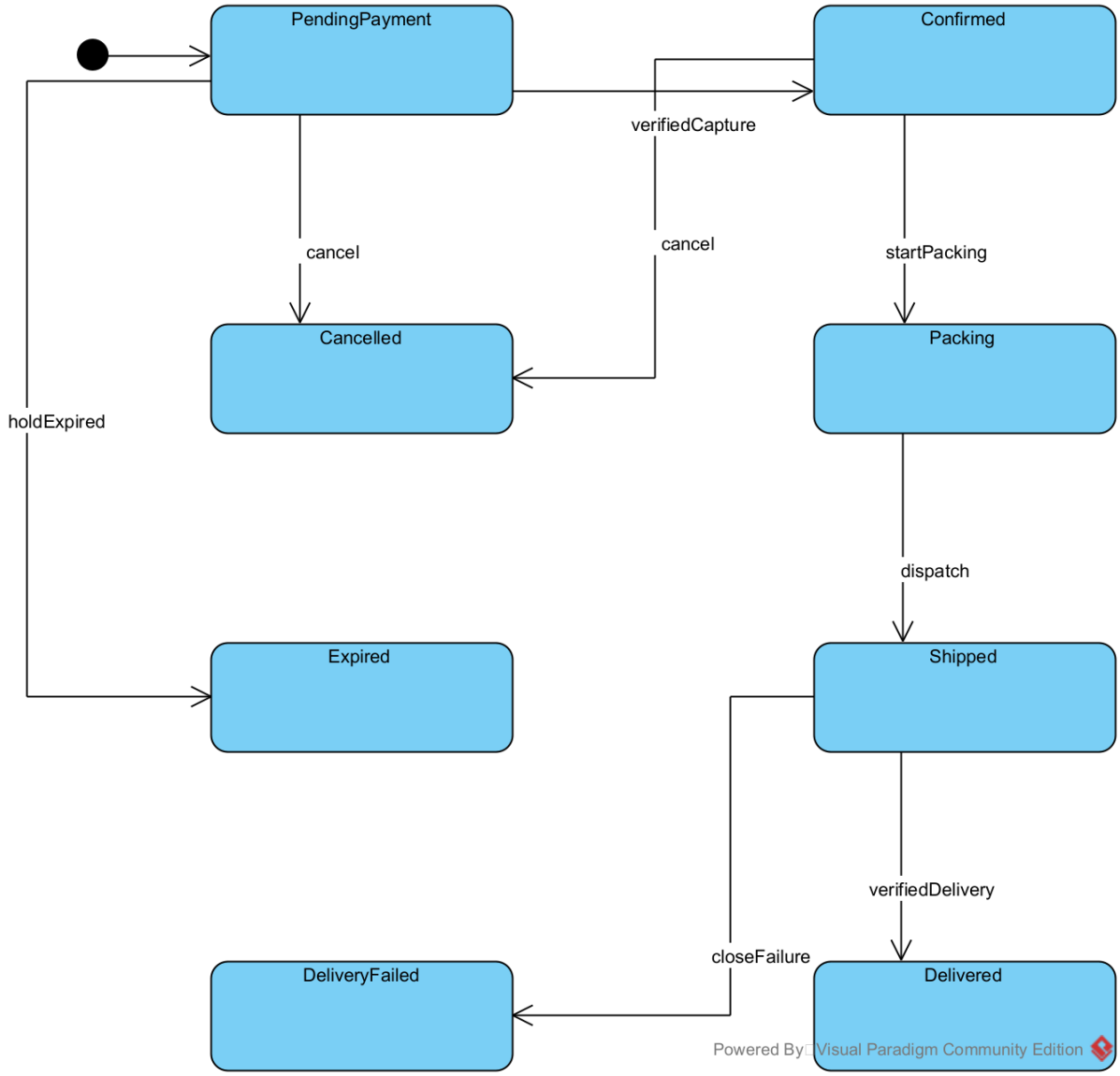


Figure 3.10: Order fulfilment lifecycle; money and returns have separate states.

Table 3.2: Order transition conditions and effects

Event	Allowed source and guard	Result and invariant
verifiedCapture	PendingPayment; matching trusted capture; hold still active	Enter Confirmed; retain allocations for fulfilment and remove the initial payment expiry condition.
startPacking	Confirmed; verified payment and sufficient active allocation	Enter Packing under a lock/version check; a competing cancellation cannot also win.

Event	Allowed source and guard	Result and invariant
dispatch	Packing; SKU checks complete; shipment reference confirmed	Enter Shipped; decrement saleable on-hand and reserved by the dispatched quantities once.
verifiedDelivery	Shipped; validated carrier evidence	Enter Delivered; store delivery time used by the return policy.
closeFailure	Shipped; Customer Service authorises permanent non-delivery using verified carrier evidence	Enter DeliveryFailed; schedule captured-funds refund independently. Consumed stock is not automatically restored.
cancel	PendingPayment or Confirmed; packing has not begun	Enter Cancelled; release active stock allocation; separately cancel unresolved payment or request refund of captured funds.
holdExpired	PendingPayment; initial 15-minute deadline passed	Enter Expired; release active holds once; reconcile provider status and compensate any late capture.

Payment status uses Pending, Processing, Succeeded, Failed, and Cancelled. Failed may be retried while the same order remains eligible; unresolved Processing is reconciled instead of treated as failure. A provider object that already succeeded cannot be moved back to Failed by an older notification. Captured money remains recorded even when fulfilment is Cancelled or Expired.

Shipment status uses Registered, InTransit, Delivered, DeliveryFailed, and ReturnedToSender. A temporary failed attempt remains InTransit while redelivery is possible. Permanent non-delivery requires verified evidence and an authorised UC12 closure; it cannot overwrite Delivered. The first-release policy refunds the full captured order amount, including delivery, for this closure. A parcel physically returned by the carrier is counted and inspected through UC10 using a unique shipment/receipt reference; only saleable units are posted back. Lost parcels produce no stock increase. This is separate from a customer's UC08 return of an already delivered order.

Refund status uses Requested, Pending, Succeeded, and Failed. Requested/Pending amounts reserve a financial entitlement. Failed means an exception needing verified resolution; retry of that refund keeps the same commitment and request key. A commitment is released only when cancellation/failure is authoritative and the business decision no longer requires the refund. Returns use Requested, Approved, Received, Inspected, AwaitingChoice, Resolved, Rejected, and Withdrawn. Split receipts may accumulate while the request remains Received; Resolved requires the chosen commercial remedy to finish. These states are independent of the original order remaining Delivered.

3.7 Domain concepts and analysis class models

The class models identify business concepts and their relationships. Identifier denotes a stable business identity; MoneyVND denotes an exact integer number of VND; DateTime denotes an instant stored consistently. They are conceptual types rather than claims about a programming language. Attributes are representative business information; the database design expands them with keys, audit, and integration fields.

Figure 3.11 establishes Product as a garment design and ProductVariant as a sellable SKU. Each variant belongs to one Product and has one InventoryBalance in the one-warehouse case. Every Product belongs to one primary Category; a category can contain many products. A SizeChart can be reused by several products, while a product has at most one applicable chart version. Accessories may have no size chart. The zero-or-more variant multiplicity permits draft products; publication separately requires at least one active, valid variant. InventoryBalance.available is a derived view of onHand minus reserved, not an independently editable stock counter.

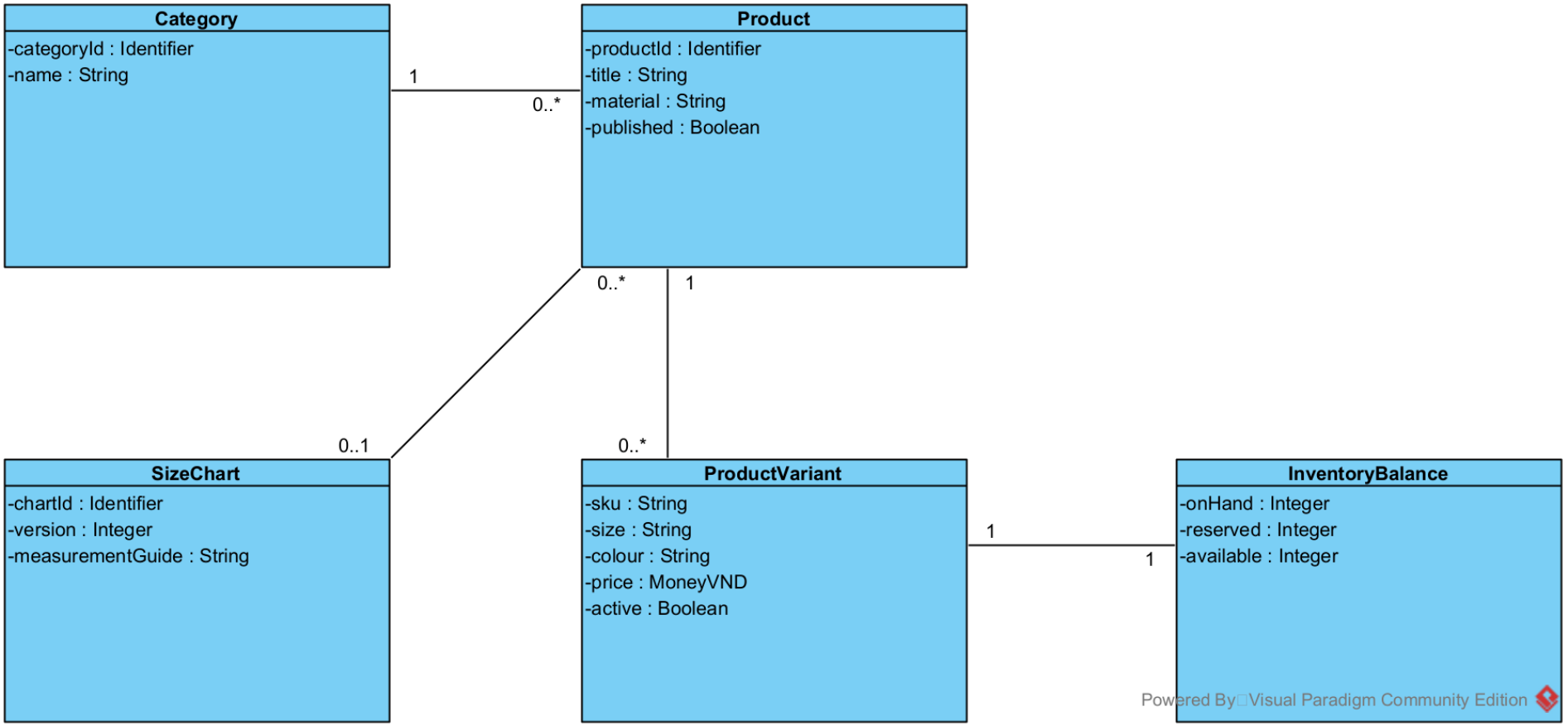


Figure 3.11: Catalogue, size charts, SKU variants, and warehouse inventory.

Figure 3.12 separates cart intention from purchase commitment. A Customer has at most one active Cart and may have many Orders. Each CartLine selects one exact ProductVariant. An Order has at least one OrderLine and exactly one AddressSnapshot, while every OrderLine keeps a link to the retained variant plus an immutable commercial description. Product edits cannot silently rewrite the purchased title, size, colour, chart version, price, discount, or delivery details.

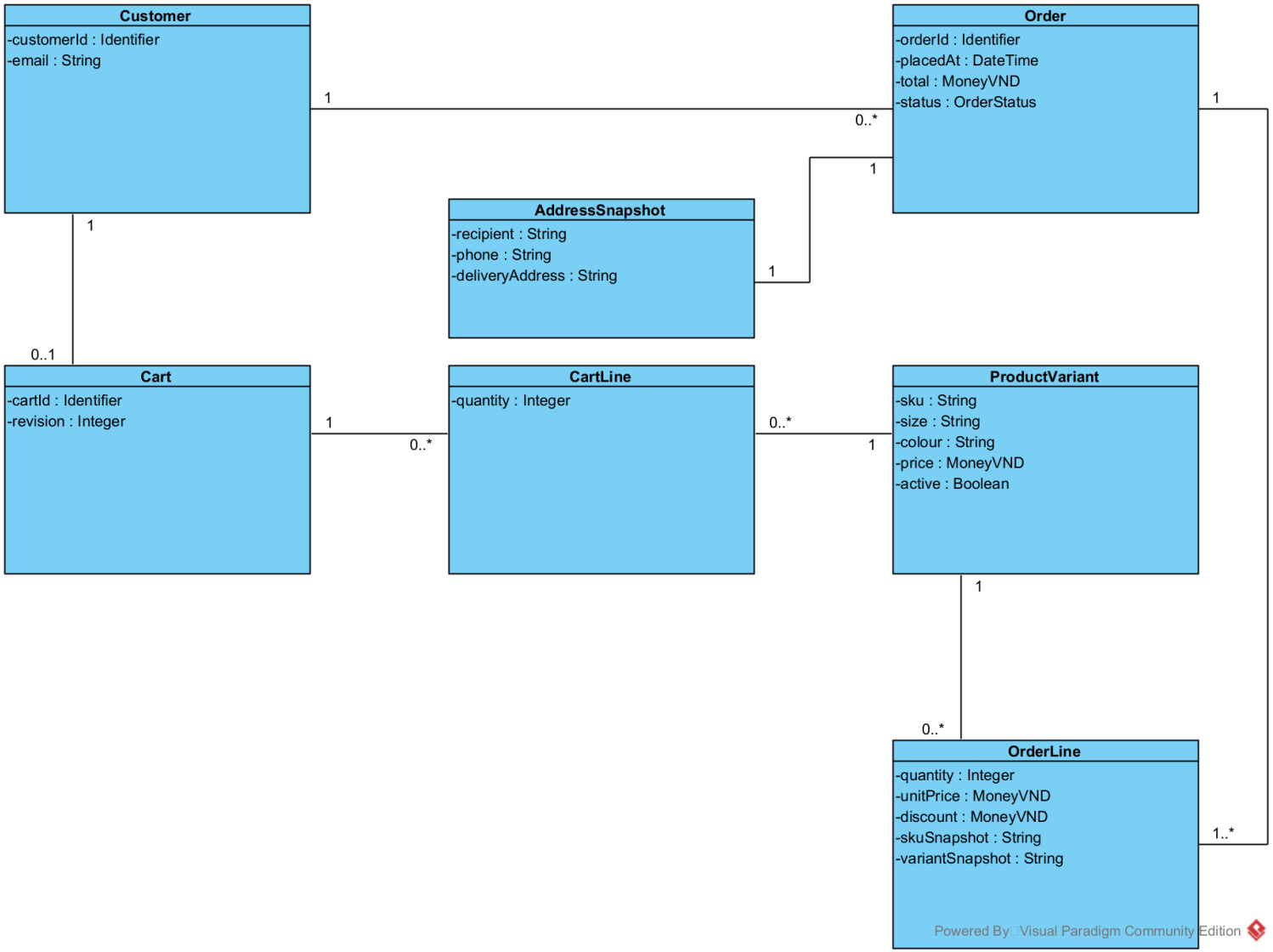


Figure 3.12: Customers, carts, committed order lines, and address snapshots.

Figure 3.13 makes stock commitment explicit. A Reservation belongs to one original Order and one SKU. Its kind distinguishes initial purchase allocation from a linked replacement allocation; a replacement additionally references its ExchangeFulfilment in persistence. Active purchase holds expire while PendingPayment; after confirmation they remain allocated until consumption or permitted cancellation. A replacement does not change the original order's Delivered state. Reservation histories are retained after release/consumption, so a historical order may have several reservations even though only its active allocations contribute to reserved stock.

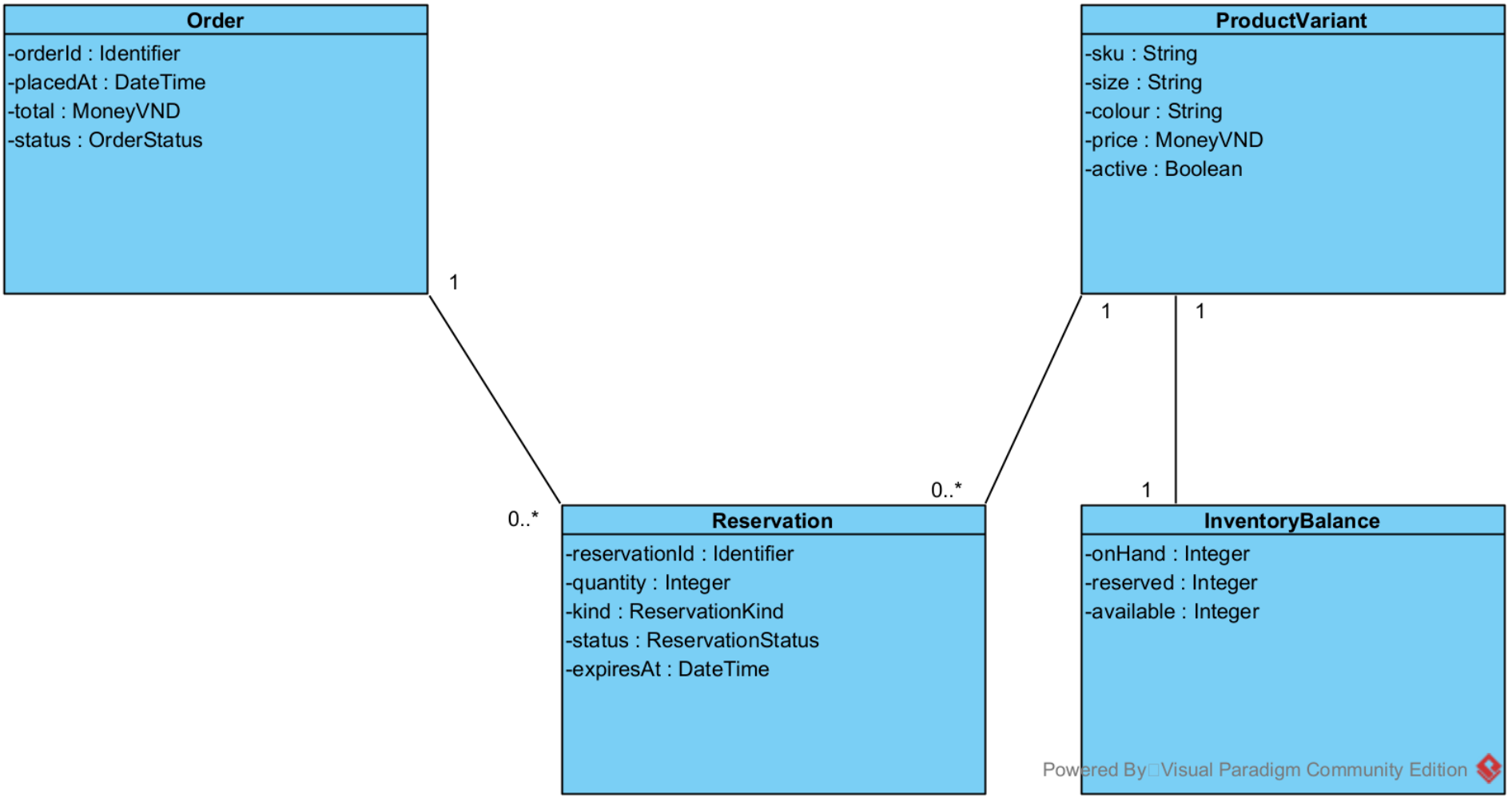


Figure 3.13: Stock reservations are explicit commitments and are independent of carts.

Figure 3.14 separates payment attempts, shipment evidence, returned quantities, and refunds. An Order can have several payment attempts, but only one successful payment object is accepted for its original payable total; further real captures require compensation. Each Refund references one Payment. An original Order has at most one Shipment in the first release. Each ReturnLine identifies one original OrderLine; a ReturnRequest groups one or more such lines. A ReturnLine may lead to one linked ExchangeFulfilment for its allocated replacement quantity. Additional persistence links identify replacement SKU, tracking, reservations, and refund allocation; they do not turn the analysis model into a diagram of every database column.

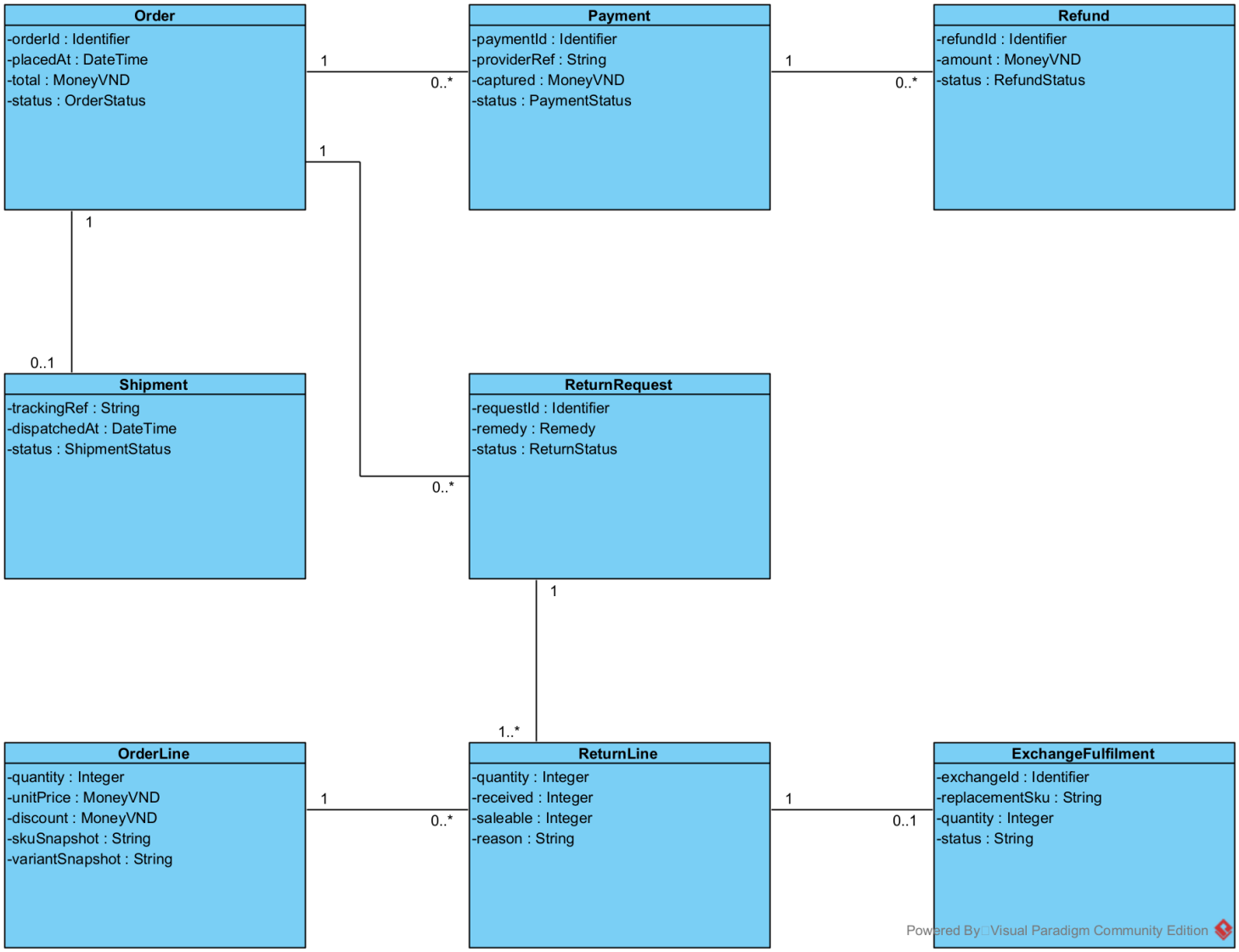


Figure 3.14: Independent payment, shipment, return, and exchange concepts.

3.8 Cross-model consistency

The shared ProductVariant, Order, and OrderLine concepts carry the same meaning in every view. UC04 and the checkout sequence both establish stock commitments before remote payment processing. The order state machine permits packing only from Confirmed; UC11 performs that guarded transition. The inspection swimlane restores stock from physical evidence, whereas UC14 changes financial state. UC08 and the class model support partial per-line returns; the quantity rules restrict the otherwise broad many-valued associations. Multiplicity alone cannot express monetary caps, cumulative return quantities, stock inequalities, or time windows, so these remain explicit business rules and transaction constraints.

Chapter 4

System Design

4.1 Architecture and responsibility allocation

The proposed implementation is a modular application with one transactional data store. This choice supports a single retailer without requiring distributed stock allocation across independent merchants. Catalogue, identity/access, cart, checkout, inventory, order fulfilment, returns, payments/refunds, and reporting form distinct application modules with explicit interfaces. Their separation supports maintainability; their shared database supports atomic changes where local business invariants span several modules.

Figure 4.1 shows dependencies rather than chronological message flow. The UI calls application interfaces over HTTPS. Application modules use the transactional repository and record durable work. A worker processes inbox/outbox entries and delegates external calls to payment and shipping adapters. Adapters translate provider-specific fields and errors into local business outcomes. Provider credentials remain server-side. Reporting queries approved source data; it cannot set stock or paid flags.

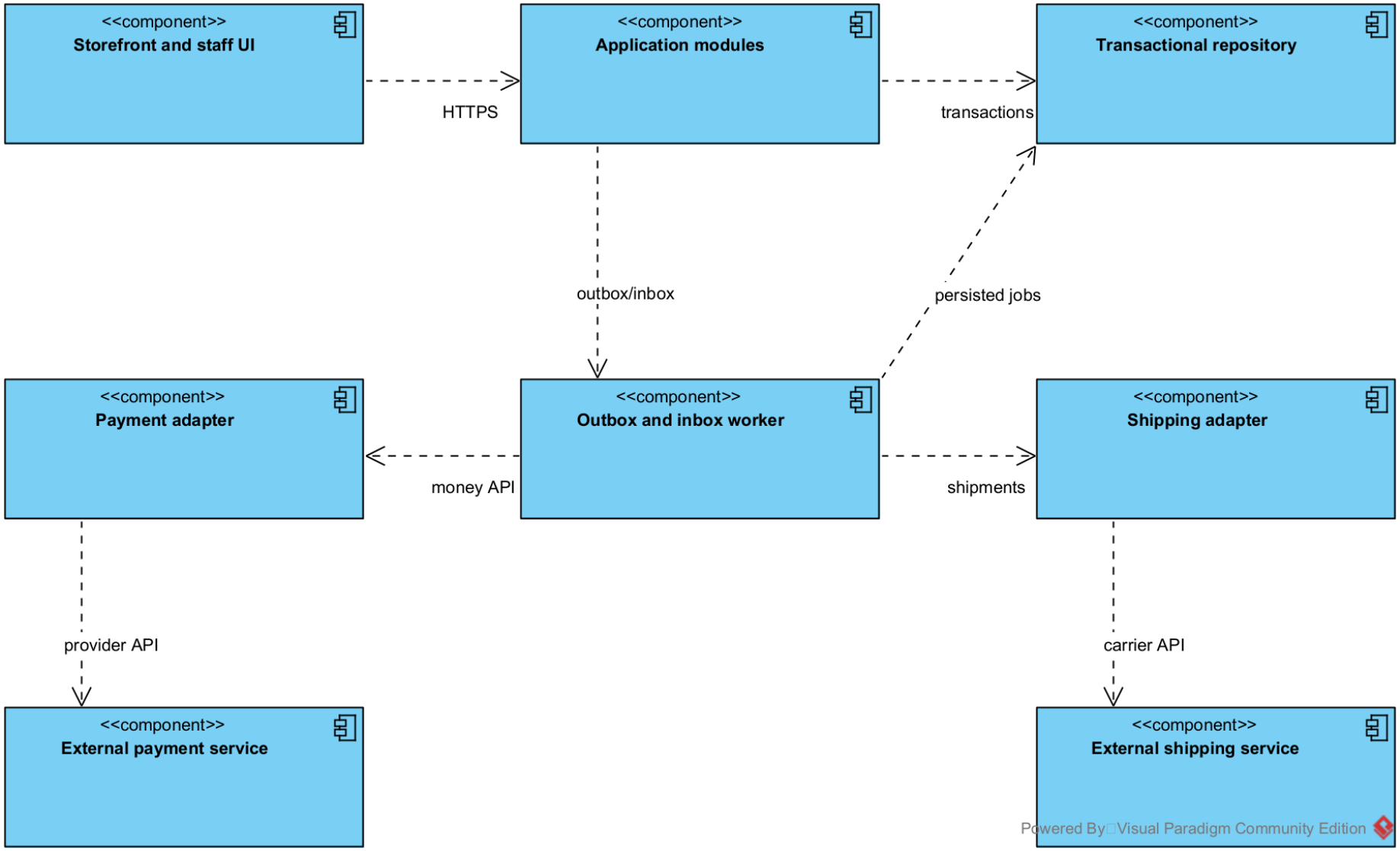


Figure 4.1: Application modules, durable work processing, and external service adapters.

Table 4.1: Design responsibilities and boundaries

Module	Responsibility	Boundary rule
Catalogue	Publish products, variants, images, and chart versions	Never rewrite purchase snapshots.
Checkout/cart	Customer cart ownership, consented quote, request deduplication	Never trust client-supplied prices or paid status.
Inventory	Receipts, reservations, release, consumption, inspected restock	All stock effects require a unique movement/allocation reference.
Orders/fulfilment	Guarded lifecycle transitions and SKU packing	Packing requires confirmed funds and active allocation.
Payments/refunds	Provider object linkage, money commitments, reconciliation	Remote timeouts remain uncertain until reconciled.
Returns/exchanges	Eligibility, quantity allocation, inspection, remedy	Financial approval is separate from physical stock receipt.
Identity/audit	Roles, resource ownership, staff provisioning, audit events	Enforce permission at every mutation/read boundary.
Worker/adapters	Retryable external calls, inbox processing, notification	Business effects are idempotent despite repeated delivery.

4.2 Checkout transaction and interaction design

Figure 4.2 is the successful checkout trace. Steps 2–5 belong to one local database transaction, including the SKU locks and final commit. Returning validated data at step 3 does not release those transaction locks. External payment initiation occurs only after commit. The customer receives PendingPayment and provider instructions, not a local assertion that payment succeeded. Execution bars are omitted because this is a responsibility-level interaction model; message order, synchronous calls, and dashed replies carry its relevant semantics.

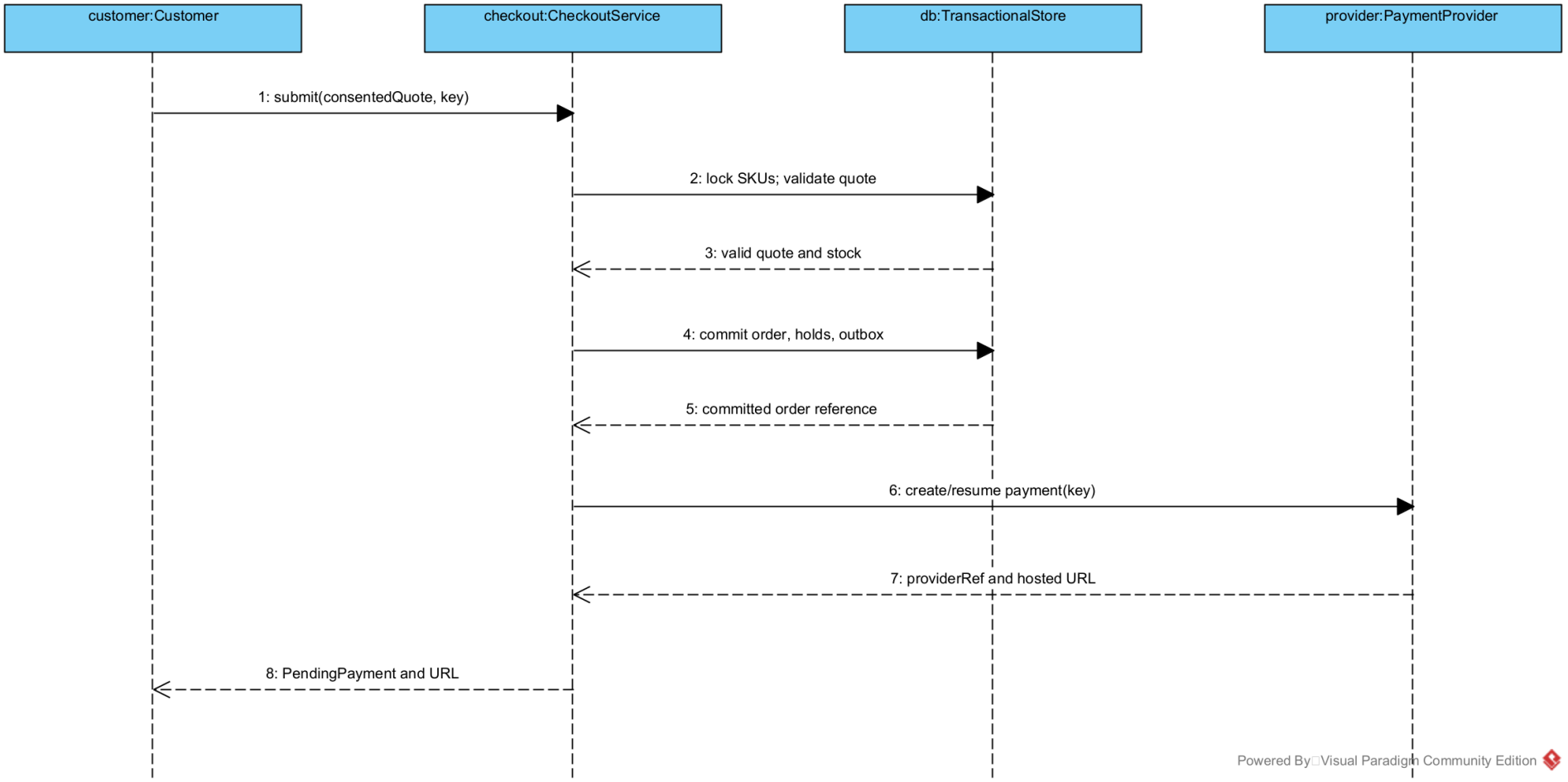


Figure 4.2: Successful checkout: commit order, stock holds, and durable work before the provider call.

The transaction loads the owned cart, verifies the consented quote and revision, locks SKU balances in sorted SKU order, and validates every requested quantity. It inserts the order, immutable lines/address, active reservations, and payment-command outbox entry together. Failure for any line rolls back every new record and stock change. The unique customer/request-key constraint returns an existing result on retries. A request fingerprint prevents a different payload from reusing the same key. Inventory does not wait on a provider network response while its rows are locked.

The payment-command worker may retry after a crash. It uses a stable key tied to the original order/payment operation and retrieves the provider object when the previous outcome is uncertain. The provider reference is durably linked to the order. An expired payment deadline is handled under the order lock; a callback racing expiry can either confirm a still-active order or record captured funds for a closed order and compensate, but cannot do both. A production adapter must define the provider's supported retry and reconciliation semantics before launch.

4.3 Provider callback and compensation design

Figure 4.3 treats receipt acknowledgment and financial processing separately. The adapter validates the original signed request, persists a unique inbox event, and acknowledges durable receipt. Message 5 is asynchronous processing of that stored event. The business update then validates provider reference, amount/currency, event identity, and the locked order state. The optional fragment applies only when the order is closed and a capture has been verified; *paid* in its guard means trusted captured funds, not a browser flag.

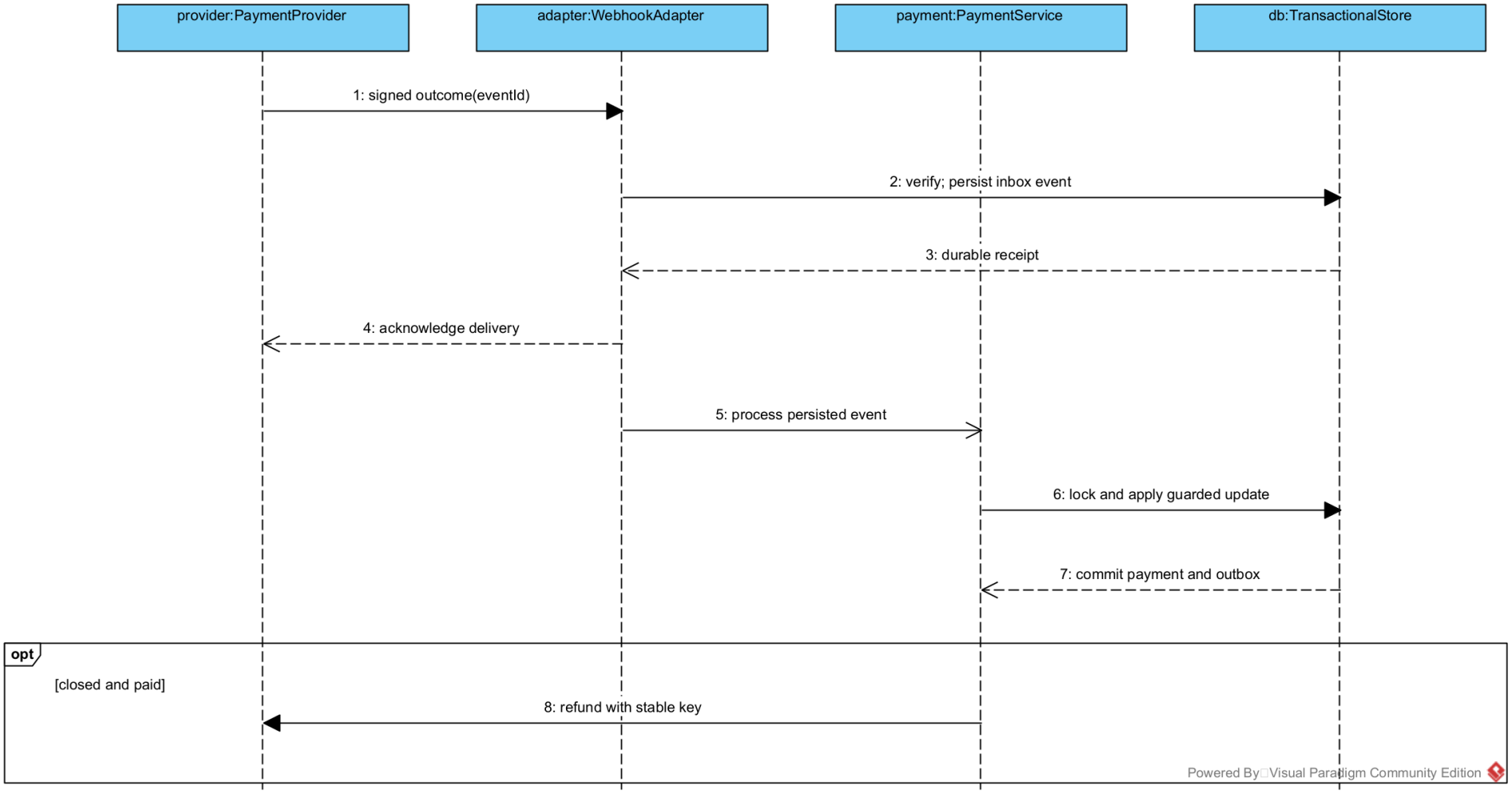


Figure 4.3: Durable callback receipt, guarded processing, and optional late-payment compensation.

Signature verification precedes inbox persistence; the step-2 message includes persistence after adapter verification, not an instruction to trust a database with cryptographic authentication automatically. A duplicate already persisted event receives an acknowledgment and does not schedule duplicate work. A distinct event describing the same provider payment is also protected by the unique provider object and guarded payment transition. Pending event delivery is retried/reconciled; timestamp order alone is insufficient to infer the final state[5].

Compensation first records a pending refund commitment and durable command in the transaction that recognises the late capture. The call shown in the optional fragment is issued through the payment adapter by retryable work after that commit. It does not mark the refund Succeeded. An unavailable provider leaves a visible pending/failed exception until verified resolution. Idempotent API requests are a provider integration pattern[6]; local unique constraints and transaction guards are still required to protect application effects.

4.4 Relational mapping and data integrity

The relational mapping below translates concepts into a persistence design. PK denotes primary key, FK denotes foreign key, and UQ denotes uniqueness. It is a design specification; no deployed database or generated application is claimed.

Table 4.2: Core relational entities and constraints

Entity	Keys and principal links	Integrity rule
Customer, Staff, Role	Customer/staff PK; email UQ; StaffRole composite PK	Public registration creates customer roles only; credentials are hashes, not plaintext.
Category, SizeChart, Product	Product PK; category FK; optional chart/version FK	Required attributes/images before publication; preserve chart versions.
ProductVariant	SKU PK; product FK; UQ(product, size, colour)	Active flag; nonnegative integer price; retain purchased variants.
InventoryBalance	SKU PK/FK; onHand, reserved; row version	Check $0 \leq reserved \leq onHand$; availability derived.
StockMovement	Movement PK; SKU FK; UQ(source type, source reference, SKU, effect)	Signed quantity and authorised reason; immutable effect history.
Cart, CartLine	Cart PK; customer UQ/FK; line UQ(cart, SKU)	Positive quantity; optimistic cart revision; no reservation FK.

Entity	Keys and principal links	Integrity rule
Order, AddressSnapshot	Order PK; customer FK; UQ(customer, request key); address order PK/FK	Immutable agreed snapshot; separate fulfilment status/version.
OrderLine	Line PK; order/SKU FKs; unique line number per order	Positive quantity; integer agreed unit price and allocated discount; preserve descriptions.
Reservation	PK; order/SKU FKs; optional exchange FK; unique operation/SKU	Positive quantity; kind, active/consumed/released state, deadline; guarded once-only effects.
Payment	PK; order FK; provider reference UQ	Integer amount/currency; immutable captured evidence; only one accepted successful object for original total.
InboxEvent, OutboxCommand	PK; UQ(provider, event ID) / UQ(operation key)	Durable attempts, state, correlation, and next retry; business commit includes new command.
Shipment, TrackingEvent	Shipment PK; order UQ/FK; tracking reference UQ; provider/event UQ	One original shipment per order; guarded delivery progression.
ReturnRequest, ReturnLine	Request/line PKs; order and original line FKs	Lock original line entitlement; positive quantities; active/completed allocation cap.
Inspection	PK; return-line FK; unique receipt-line reference	$0 \leq \textit{saleable} \leq \textit{accepted} \leq \textit{received}$; cumulative receipts cannot exceed approval.
Exchange fulfilment	PK; original return-line FK; replacement SKU FK; tracking reference	One linked replacement allocation per accepted return line; equal product and agreed price.
Refund, RefundAllocation	Refund PK; payment FK; request key UQ; allocation line FK	Pending and successful commitments count toward payment/line caps.
AuditEvent, Notification, Review	PKs; business reference; notification operation UQ; review UQ(customer, order, product)	Append-only audit; independent delivery retry; purchase-checked moderated review.

Foreign keys prevent orphan lines and references; restricted deletion protects transaction history. Cross-row sums require transactional enforcement rather than a simple row CHECK. For instance, an original OrderLine is locked before allocating a new return quan-

tity, and a Payment is locked before adding a refund commitment. The stored reserved balance and active Reservation sum are reconciled periodically; application transactions update them together. Useful indexes cover active catalogue filters, customer/order history, SKU movements, unresolved payment/refund state, and due inbox/outbox work. Search indexes are read models and never authorise stock allocation.

4.5 Stock and money examples

Consider a SKU with $onHand = 5$ and $reserved = 0$. Reserving two units gives $(5, 2)$ and available stock 3. Dispatch gives $(3, 0)$ and still available stock 3. Cancelling before dispatch would instead produce $(5, 0)$. If two dispatched units return and only one is saleable, inspection restores one unit, producing $(4, 0)$; refunding two units does not independently add two units to stock. A last-unit race must allocate at most one successful hold even if both carts previously showed availability.

For money, suppose three units cost 399,000 VND each, the agreed line discount is 70,000 VND, and delivery is 30,000 VND. Gross merchandise is 1,197,000 VND; net merchandise is 1,127,000 VND; the order total is 1,157,000 VND. Allocate the discount deterministically across stable unit ordinals as 23,334, 23,333, and 23,333 VND. The corresponding refund entitlements are 375,666, 375,667, and 375,667 VND. Refunding the first unit commits 375,666 VND; it does not recompute a larger refund from the current catalogue price. A second concurrent request cannot allocate that same unit ordinal again. Remainder allocation guarantees that full item refunds sum exactly to the agreed discounted merchandise value. The baseline policy excludes delivery refunds for post-delivery item returns; a full pre-dispatch cancellation or permanent non-delivery closure includes the agreed delivery charge.

4.6 Interface behaviour and access design

Product pages show the chosen size/colour explicitly, retain size-chart context, and disable unavailable selections with text rather than colour alone. Cart estimates are labelled as estimates. Checkout repeats the final SKU quantities, recipient snapshot, delivery charge, and payable total before consent. A changed quote is shown for reconfirmation, not silently accepted. Pending payment screens include an order reference, resumable provider instruction, and status refresh; their copy avoids a false success message.

Order detail separates payment, delivery, return, and refund status. Return forms list eligible order lines and remaining quantities; unavailable exchange stock creates a clear choice rather than an automatic unwanted refund. Warehouse screens require SKU identity and quantities, and distinguish saleable from damaged units. Financial exceptions show provider reference, verified amount, pending commitments, and reconciliation history. All forms provide field-level errors and preserve safe input after validation failure. Core journeys target labelled controls, keyboard operation, visible focus, and accessible status messages under WCAG 2.2 AA[8]; this is an acceptance target, not a certification claim.

Server-side checks apply on every relevant request. Customers access only owned carts,

addresses, orders, and requests. Merchandisers change catalogue data; Warehouse Staff change authorised stock/packing/inspection records; Customer Service makes permitted cancellation and return decisions; Finance Staff control refunds/reconciliation; Managers read approved aggregates; Administrators provision access. Staff use MFA. Hosted payment interfaces keep raw card credentials out of application storage. TLS, secure sessions, password hashing, input validation, parameterised queries, CSRF protection for browser mutations, and verified provider signatures are design controls reviewed using ASVS[7]. Webhook signature verification replaces browser-session trust for external callbacks. Secrets are kept out of source, customer pages, and logs.

4.7 Deployment, recovery, and operational reporting

Figure 4.4 shows logical deployment nodes, not a claim of purchased infrastructure. The reverse proxy is the public HTTPS entry point. Application and worker hosts reach the primary database through private authenticated connections. The worker calls provider APIs; signed callbacks return through the proxy. Backup storage is access-restricted and separate from the primary. Application/worker processes can initially share a machine while retaining separate credentials and execution roles; physical sizing follows load testing.

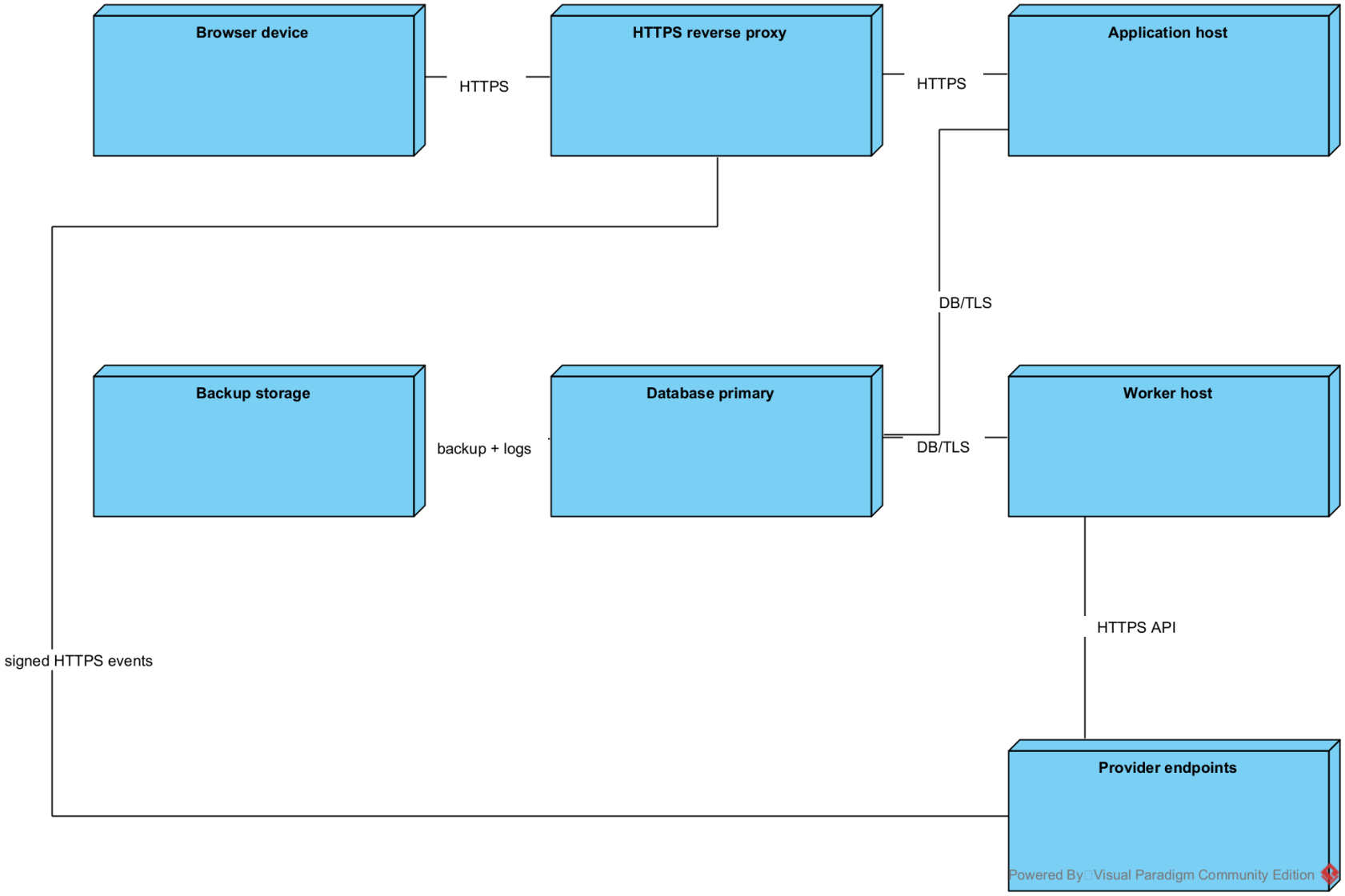


Figure 4.4: Deployment topology, provider callbacks, and backups.

Backups plus database log recovery target the proposed 15-minute RPO and four-hour RTO. A restore drill verifies data, outstanding reservations, inbox/outbox replay, and provider reconciliation. External payment outcomes may exist beyond the recovered local point; restoration must query provider references before repeating charges/refunds. Audit logs include business reference and correlation identifier, but exclude credentials and unnecessary personal data. Retention and deletion schedules require approved operational/legal policy; customer profile erasure does not justify corrupting required financial evidence.

Reports distinguish order value, captured funds, successful refunds, and net cash movement. A daily order-value measure uses placedAt and agreed totals; captured funds use verified capture time; refunded funds use provider-confirmed refund completion time. Net cash movement is captured funds minus successful refunds for the selected event period and is not labelled profit or recognised revenue. SKU units sold exclude cancelled/unpaid orders and use a disclosed dispatch or delivery basis; return rate states its unit-based numerator and delivered-unit denominator. Stock reports show saleable on-hand, reserved, available, and damaged counts separately. Use the Asia/Bangkok reporting timezone consistently for day boundaries while storing timestamps as UTC instants.

Chapter 5

Traceability and Validation

5.1 Validation approach and evidence status

Validation asks whether the proposed system addresses stakeholder needs and whether its models agree with business rules. Verification asks whether particular obligations can be checked against an implementation or artifact. This report completes a design review and artifact verification: native UML exports were inspected, report compilation was checked, and the traceability below covers the requirement baseline. The acceptance scenarios are specifications for future implementation testing; they are not reported as executed production tests.

Each functional requirement has an origin, actor goal, applicable rule, design location, and observable acceptance outcome. The business problem codes P01–P07 and stakeholder needs N01–N10 are defined in Chapter 2. Where no current defect is necessary to justify a supporting feature, its stakeholder need remains sufficient origin evidence. Must requirements are release gates; Should and Could requirements retain traceability even if their implementation is deferred.

5.2 Functional requirement traceability

Table 5.1: Complete functional traceability and acceptance outcomes

FR	Origin	Use case / rules	Model or design	Acceptance scenario and expected result
FR01	N01; P01	UC01; BR01	Fig. 3.2, 3.11	AT01: Filter blue/M; returned products have a matching variant. Unavailable combinations cannot be added as available purchases.

FR	Origin	Use case / rules	Model or design	Acceptance scenario and expected result
FR02	N01,N07	UC02; BR14	Account and interface design	AT02: Register, sign in, edit owned address, sign out; another account's address and staff role elevation are denied.
FR03	N01; P01,P02	UC03; BR02	Fig. 3.12	AT03: Add two exact SKUs, change/remove quantity; no InventoryBalance or Reservation changes. Stale cart revisions are rejected.
FR04	N01; P03	UC04; BR04–05	Fig. 4.2	AT04: Changed quote requires consent again. Same request key yields one order; same key with different payload is refused.
FR05	N03; P02	UC04; BR02–03	Fig. 3.13	AT05: Two buyers compete for the last unit: only one reservation succeeds. A shortage on a second line leaves no partial order/hold.
FR06	N08; P06	UC04; BR06	Fig. 4.2	AT06: Retry an uncertain payment-create response; provider key/reference is reused and no second charge object is created blindly.
FR07	N05; P04,P06	UC05; BR03,06	Fig. 4.3	AT07: Forged/mismatched events cannot confirm an order. Duplicate events cause one effect. Late capture on an Expired order schedules compensation.
FR08	N01,N04	UC06; BR13–14	Order interface and snapshots	AT08: Customer sees separate payment/delivery/refund status for owned orders; guessed foreign order identifiers reveal no details.
FR09	N01,N04	UC07; BR07–08	Fig. 3.10	AT09: Cancel a Confirmed order before packing; release once. In a packing/cancel race only one transition wins; after Packing cancellation is denied.

FR	Origin	Use case / rules	Model or design	Acceptance scenario and expected result
FR10	N01,N04; P06	UC08; BR09	Fig. 3.14	AT10: Request one eligible delivered unit; concurrent requests cannot exceed remaining quantity. An out-of-window request gives a policy explanation.
FR11	N02; P01,P03	UC09; BR01,04	Fig. 3.11	AT11: Duplicate SKU/size-colour tuple or incomplete publication is refused. Changing a live price leaves old line snapshots unchanged.
FR12	N03,N10; P02	UC10; BR02	Fig. 3.7	AT12: Valid receipt adds saleable units and one movement. Repeated receipt does not add twice; downward adjustment below reserved stock is refused.
FR13	N03; P01,P04	UC11; BR06–08	Fig. 3.10	AT13: Unpaid order cannot pack. Wrong-SKU scan blocks dispatch. Repeated dispatch does not consume stock twice or create a second shipment.
FR14	N09; P07	UC12; BR13,15	Shipment/event data design	AT14: Signed delivery advances once; old transit cannot undo Delivered. Temporary failure permits retry; authorised permanent failure closes Shipped and schedules a full refund once. Carrier return restocks only inspected saleable goods once; lost goods do not. Unknown references enter exceptions.
FR15	N03,N04; P05	UC13; BR10–11	Fig. 3.8, 3.9	AT15: Return two units, only one saleable: restock one. Replayed receipt has no second effect; unavailable exchange stock prompts customer choice.

FR	Origin	Use case / rules	Model or design	Acceptance scenario and expected result
FR16	N05; P06	UC14; BR12–13	Refund mapping and arithmetic	AT16: Concurrent refund attempts cannot exceed captured/line entitlement. Timeout remains pending; only verified completion marks Succeeded.
FR17	N06; P07	UC15; BR13	Reporting definitions	AT17: Place/capture/refund on different days; each appears in its defined event period. Net cash is not labelled profit or order value.
FR18	N07; P06	UC16; BR14	Role/audit design	AT18: Exercise each role and denied action; sensitive effects have actor/event references. Revoked staff access and cross-customer reads are refused.
FR19	N01,N04	UC17; BR13	Durable outbox design	AT19: Notification delivery fails after commit: the order remains confirmed, notice is retried, and authoritative status remains readable.
FR20	N01,N06	UC18; BR14	Review mapping	AT20: Delivered buyer submits once; unpublished moderation queue retains it. Unpurchased product and duplicate order/product review are refused.

5.3 Nonfunctional verification plan

Table 5.2: Nonfunctional requirement evidence to collect before release

NFR	Design support	Required acceptance evidence
NFR01	Catalogue indexes; short local allocation transaction	NFT01: Test 100 concurrent sessions against 10,000 variants with a published workload mix and hardware. Record internal p95 search ≤ 2 s and allocation ≤ 3 s; disclose provider time separately.

NFR	Design support	Required acceptance evidence
NFR02	Monitored proxy/application/worker and separate provider incident classification	NFT02: Measure successful core journeys over a calendar month and verify the 99.5% service target under the defined maintenance exclusions.
NFR03	Locks, versions, uniqueness, inbox/outbox, commitment caps	NFT03: Run AT05, AT07, AT09, AT10, AT15, and AT16 concurrently with retries/crashes; reconcile balances, allocations, and monetary effects.
NFR04	Server authorisation, staff MFA, secure sessions/credentials, callback signatures	NFT04: Security review of allowed/denied role and ownership cases, session revocation, injection handling, and invalid signatures against the selected ASVS controls.
NFR05	Hosted payment UI; minimised contact data; controlled logs	NFT05: Inspect storage, backups, logs, and customer responses for raw card credentials, secrets, unnecessary personal information, and excessive access.
NFR06	Labelled controls, keyboard flows, visible focus and textual statuses	NFT06: Manual keyboard and screen-reader journeys plus automated accessibility checks on browsing, cart, checkout, and returns; evaluate applicable WCAG 2.2 AA criteria.
NFR07	Isolated backups, database logs, replay/reconciliation procedure	NFT07: Restore a representative failure within 4 h, demonstrate no more than 15 min data-loss exposure, and reconcile outstanding provider operations before replay.
NFR08	Module/adaptor separation; policy versioning; correlation IDs	NFT08: Replace a provider test adapter without changing stock rules; trace one checkout and one partial return across API, transaction, and worker evidence.
NFR09	Append-only actor/event audit and unique movement/financial references	NFT09: Reconstruct every effect in sampled stock, cancellation, inspection, and refund histories; detect any missing or unauthorised reference.

5.4 Critical scenario walkthroughs

5.4.1 Last unit and multi-line atomicity

Two customers both observe one available M/blue SKU. Their carts do not allocate it. Checkout A obtains the inventory lock and reserves it; checkout B subsequently observes zero availability and fails without creating a purchase commitment. If A also requests an

unavailable second SKU, A rolls back the first allocation before releasing its locks. This outcome depends on a transaction covering every line, not on a stock check performed before the transaction.

5.4.2 Payment capture versus hold expiry

Callback processing and expiry both lock the same order. If a valid callback confirms the still-active allocation first, expiry sees Confirmed and does not release it. If expiry closes and releases first, the callback records real capture evidence but cannot revive the order or promise released stock. The durable compensation commitment prevents the retailer retaining unexplained customer funds. Loss of a callback is handled through provider reconciliation; a browser redirect cannot substitute for it.

5.4.3 Cancellation versus packing

A customer and warehouse worker concurrently request mutually exclusive transitions from Confirmed. The first committed guarded change wins. A successful cancellation releases allocations and prevents later packing. Successful entry into Packing prevents customer cancellation under the agreed policy. Any captured-money refund is tracked independently and must not rewrite the packing/physical-delivery facts.

5.4.4 Partial returns, discounts, and duplicate receipts

A customer requests two of three delivered units, then submits the same request again. The remaining quantity is computed under the original line lock and allocated only once. Inspection may accept two units but classify only one as saleable, producing one stock restoration. Unit-ordinal refund allocation preserves the original discount remainder. Replayed receipt and provider refund callbacks cannot independently repeat either stock or money effects.

5.5 Requirement quality review and change control

Stable identifiers, actors, resources, and observable outcomes make the requirements reviewable. Explicit load profiles and trust checks replace vague speed/security claims. Numeric targets remain proposed; optional reviews do not block stock, order, and refund correctness. Seller payouts, manufacturing, and expiry inventory remain outside scope.

The return window and exchange policy require retailer approval; UML defines notation. Provider documentation supports patterns, not local market availability. Changes must update rules, use cases, diagrams, data constraints, and acceptance scenarios together. Artifact verification does not establish application performance, connectivity, or customer acceptance.

Chapter 6

Conclusion and Limitations

Atelier Fashion requires consistent SKU identity, immutable purchase evidence, and controlled coordination between stock and money. The investigation links business problems and stakeholder needs to a prioritised requirement baseline. Actor/use-case views distinguish customer goals from internal responsibilities and external provider interfaces. Activity models establish validation and physical inspection, while the order state machine and independent payment/refund states prevent a single status flag from concealing incompatible business facts. Analysis classes distinguish products from variants, carts from reservations, and original order lines from returned quantities.

The proposed design preserves local transaction integrity before external calls and uses durable event/command processing for retries and reconciliation. Exact integer money and deterministic discount allocation support partial refunds. Access and audit controls protect stock and financial decisions, and the complete traceability matrix provides observable release criteria. Native editable Visual Paradigm diagrams and the LaTeX source make the design reviewable and maintainable beyond the PDF.

The study is limited by its fictional business scenario and absence of field elicitation, cost quotations, production workloads, implemented software, and provider acceptance tests. One warehouse, one original shipment, prepaid online payments, and constrained exchanges simplify the first release. A real deployment requires confirmed policies, jurisdictional tax/privacy review, chosen provider contracts, staff training, migration reconciliation, and implementation acceptance testing. Future work may introduce guest checkout, multiple warehouses/shipments, broader exchange pricing, promotions, and richer fit guidance only after revisiting the same consistency and traceability obligations.

Appendix A

Visual Paradigm Modelling and Reproduction Guide

A.1 Native project and workflow

The editable project is `diagrams/AtelierFashion.vpp`, created in Visual Paradigm 18 Community Edition. Every report diagram is composed of native UML shapes and relationships in that application. The local Open API plugin creates the models, arranges their bounds, saves the project, and invokes Visual Paradigm's own PNG export at scale 2. No diagram is substituted by PlantUML, Mermaid, drawing-library output, or an externally generated raster. Colour, font, and connector-style settings are left at the application's defaults; only positions, dimensions, caption placement, and necessary UML semantics are adjusted. The Community Edition watermark is retained.

For manual editing, open the VPP file, select the named diagram in Diagram Navigator, edit model properties and shape bounds, and save. Export with Project > Export > Diagrams as Image, choosing PNG. Inspect the exported image as well as the in-application canvas: labels must be complete, nodes must not overlap, connectors must reach their intended endpoints, and unrelated connectors/labels must not collide. Review the inserted image again at PDF page size. Wide class and interaction diagrams use landscape report pages for legibility.

A.2 Purpose, content, and construction by diagram

Table A.1: Diagram construction and review guide

Export	Purpose and content	Visual Paradigm construction/review procedure
01 customer use cases	Customer goals and payment boundary	New Use Case Diagram; add System, Visitor, Customer, Payment Provider and ellipses; place ellipses inside the system. Add Customer generalisation toward Visitor, actor associations, and Place order include arrows toward validation, reservation, and payment initiation. Inspect all arrowheads and captions.
02 operations	Catalogue, stock, fulfilment, inspection, carrier events	Add staff/provider actors outside the same system boundary and UC09–UC13 ellipses inside. Associate the carrier with shipment support and its event goal. Route the provider link around adjacent use cases and check that every line touches its ellipse.
16 back office	Cancellation, returns, refunds, reports, access	Use Case Diagram with Customer Service, Finance, Manager, Administrator and Payment Provider. Associate each role with its permitted goals. Check that the finance/report fan does not cross unrelated ellipses.
18 supporting goals	Notifications, optional reviews, financial reconciliation	Reuse Customer, Payment Provider, and UC05; add UC17 and UC18 and Finance Staff's reconciliation participation. Keep actors outside the system and distinguish the optional review feature in the requirements.
03 search	Variant filters and unavailable/no-match retries	New Activity Diagram; use Initial Node, Action, Decision Node, Merge Node and Activity Final Node. Place mutually exclusive guards on decision outputs. Route retry paths to explicit merges before re-entering actions. Verify both labels and control semantics.
04 checkout	Customer, system, and provider responsibilities	Activity Diagram with three vertical swimlane partitions. Put consent in Customer, local allocation in System, hosted payment in Provider, and verified reconciliation back in System. Review lane ownership and committed/rejected branches.

Export	Purpose and content	Visual Paradigm construction/review procedure
05 inspection	Physical receipt before stock restoration	Three partitions: Customer Service, System, Warehouse. Place approval-reference check, physical inspection, receipt validation, and once-only restock in their responsible lanes. Give invalid/valid receipt branches separate visible destinations.
14 receiving	Validated, auditable stock receipt	Activity Diagram with receipt identification, validation, guarded posting, duplicate/invalid alternative and final nodes. Confirm the invalid branch never passes through the stock-posting action.
15 remedy	Refund versus reserved exchange	Activity Diagram with remedy and stock decisions. Add explicit refund, exchange, allocated/unavailable guards. Link successful allocation to replacement fulfilment; unavailable stock ends the attempt awaiting customer choice.
06 order states	Permitted fulfilment lifecycle	New State Machine Diagram; use Initial Pseudostate, State and Transition. Add PendingPayment, Confirmed, Packing, Shipped, Delivered, Cancelled, Expired and DeliveryFailed. Use short event captions and the report transition table for guards/effects. Route expiry and permanent failure outside state boxes.
07 catalogue classes	Category, Product, Variant, SizeChart, InventoryBalance	New Class Diagram; add representative attributes and associations with multiplicities. Keep product/variant distinct and optional chart use explicit. Space multiplicity labels away from class corners.
08 shopping/order classes	Customer, Cart, CartLine, Order, OrderLine, AddressSnapshot and Variant	Reuse existing model classes when adding views. Associate cart lines with exact SKU and committed orders with snapshots. Route Order–OrderLine outside the Variant box; place AddressSnapshot where its link avoids the cart/SKU connector.

Export	Purpose and content	Visual Paradigm construction/review procedure
17 reservations	Explicit commitment separate from carts	Class Diagram reusing Order, Variant and InventoryBalance; add Reservation with quantity, kind, state and deadline. Associate each reservation with one original order and one SKU. Check that diagonal associations do not intersect the inventory link.
09 after-sales classes	Payment, Refund, Shipment, ReturnRequest, ReturnLine and ExchangeFulfilment	Reuse Order and OrderLine. Show one payment per refund, optional original shipment, per-line returns and optional exchange fulfilment. Interpret quantity/money caps through rules, not multiplicity alone.
10 checkout sequence	Commit local work before provider interaction	New Sequence Diagram; add customer, checkout service, transactional store and provider lifelines. Use synchronous calls and dashed replies in vertical time order. Steps 2–5 share a local transaction; no database lock extends into the provider call.
11 callback sequence	Durable receipt, guarded update, optional compensation	Add provider, webhook adapter, payment service and store lifelines. Acknowledge persisted receipt; use an asynchronous worker message; add an opt fragment guarded by closed and paid for compensation. Check guard text against lifelines and message labels.
12 components	Application and integration dependencies	New Component Diagram; add UI, application, repository, worker, adapters and external services; add directed dependencies. Use concise labels that fit gaps. Verify actual component names after saving, then export.
13 deployment	HTTPS entry, private data access, backup, provider callbacks	New Deployment Diagram; add native Node shapes and communication associations. Route callbacks around other nodes and connections; keep all link labels within export bounds.

A.3 Local automation and report rebuild

The modelling source is in `automation/FashionPlugin.java`; diagram specifications and generators are in `automation/`. The plugin runs inside Visual Paradigm and consumes a tab-separated job file. Completion and errors are written separately; a generator exit is not itself proof that Visual Paradigm completed a diagram. After each completed export, the image is reviewed and any defects are corrected before the next model is accepted. The native model audit records diagram/model identifiers and associations independently of the PNGs.

The report source is `report/main.tex` with separate chapter files. From the report directory, run the locally provided Tectonic executable:

```
..\automation\tectonic\tectonic.exe --keep-logs main.tex
```

Tectonic retrieves required LaTeX packages on its first run and caches them. The source archive excludes tool binaries; install Tectonic and run `tectonic main.tex` when rebuilding elsewhere. A standard TeX installation may alternatively build the same source with repeated pdfLaTeX runs until references stabilise. Keep the sibling `diagrams` folder because figures are included by relative paths. The final review checks compilation messages, reference resolution, page layout, and actual rendered figures; the report does not equate this artifact check with implementation testing.

Appendix B

Source Use and Assumption Register

B.1 Use of the supplied examples

The supplied files are retained unchanged in `third_party`. Their section structures informed coverage, while their domain choices were assessed against the retailer boundary. The fashion example suggested variant, sizing, and after-sales concerns; the marketplace example highlighted the need to distinguish stock commitments, transaction status, and fixed order evidence. Independent seller payouts and perishable expiry management were excluded. Wording, student identity, and diagrams from those reports were not adopted as the final report's own artifacts. They are local examples, not authoritative standards.

B.2 Theory and commercial references

OMG UML 2.5.1 supplies modelling notation. Shopify's public product/variant model supports the design/SKU distinction as an industry example. Stripe's payment, webhook, and idempotency documentation informs the handling of uncertain outcomes and repeated/out-of-order messages. OWASP ASVS and W3C WCAG 2.2 define review references for security and accessibility targets. Visual Paradigm's own Open API documentation supports native model creation and export. These references are cited where their ideas are used; the proposed policies and architecture remain the author's case-design decisions.

B.3 Assumptions to confirm

One retailer/warehouse/currency, authenticated checkout, online prepaid payment, one original shipment, tax-inclusive listed prices, 15-minute initial holds, the 14-day return request window, disclosed return exclusions, equal-price same-product exchanges, shipping refund policy, and numerical service targets are case assumptions. The study does not establish legal eligibility, provider market availability, realised business improvement, production performance, or completed stakeholder interviews. Validation with the actual retailer must confirm or revise these assumptions before implementation.

References

[1] Object Management Group. *Unified Modeling Language, Version 2.5.1*. 2017. https://www.omg.org/spec/UML/2.5.1.

[2] Shopify. *ProductVariant: GraphQL Admin API*. Accessed 14 October 2026. https://shopify.dev/docs/api/admin-graphql/latest/objects/ProductVariant.

[3] Shopify. *InventoryItem: GraphQL Admin API*. Accessed 14 October 2026. https://shopify.dev/docs/api/admin-graphql/latest/objects/InventoryItem.

[4] Stripe. *The Payment Intents API*. Accessed 14 October 2026. https://docs.stripe.com/payments/payment-intents.

[5] Stripe. *Webhooks*. Accessed 14 October 2026. https://docs.stripe.com/webhooks.

[6] Stripe. *Idempotent requests*. Accessed 14 October 2026. https://docs.stripe.com/api/idempotent_requests.

[7] OWASP Foundation. *Application Security Verification Standard*. Accessed 14 October 2026. https://owasp.org/projects/asvs.

[8] World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/.

[9] Visual Paradigm. *Create Use Case Diagram using Open API; Export active diagram as image via Open API*. https://knowhow.visual-paradigm.com/openapi/use-case-diagram/; https://knowhow.visual-paradigm.com/openapi/export-diagram-img-plugin/.